Chapter 4: Design

4.1.Designing Databases:

Introduction; Database Design; Relational Database Model;

Normalization; Transforming E-R Diagrams Into Relations;

Merging Relations; Physical File and Database Design;

Designing Fields;Designing Physical Tables

4.2. Designing Forms and Reports:

Introduction; Designing Forms and Reports; Formatting Forms and Reports; Assessing Usability

4.3.Designing Interfaces and Dialogues:

Introduction; Designing Interfaces and Dialogues; Interaction Methods and Devices; Designing Interfaces; Designing Dialogues; Designing Interfaces and Dialogues in Graphical Environments

Designing Database

- **Database Design** is a collection of processes that facilitate the designing, development, implementation and maintenance of enterprise data management systems.
- Properly designed database are easy to maintain, improves data consistency and are cost effective in terms of disk storage space.
- The database designer decides how the data elements correlate and what data must be stored.
- The main objectives of database design in DBMS are to produce logical and physical designs models of the proposed database system.
- **Logical model** - This stage is concerned with developing a database model based on requirements. The entire design is on paper without any physical implementations or specific DBMS considerations.
- **Physical model** - This stage implements the logical model of the database taking into account the DBMS and physical implementation factors.

- Logical Database Design
  - Based upon the conceptual data model
  - Four key steps
    1. Develop a logical data model for each known user interface for the application using normalization principles.
    2. Combine normalized data requirements from all user interfaces into one consolidated logical database model (view integration).
    3. Translate the conceptual E-R data model for the application into normalized data requirements.
    4. Compare the consolidated logical database design with the translated E-R model and produce one final logical database model for the application.
- Physical Database Design
  - Based upon results of logical database design
  - Key decisions
    1. Choosing storage format for each attribute from the logical database model
    2. Grouping attributes from the logical database model into physical records
    3. Arranging related records in secondary memory (hard disks and magnetic tapes) so that records can be stored, retrieved and updated rapidly
    4. Selecting media and structures for storing data to make access more efficient

Deliverables and Outcomes

- Logical database design
  - must account for every data element on a system input or output
  - normalized relations are the primary deliverable
- Physical database design
  - converting relations into database tables

Relational Database Models

- **Relational Database**: data represented as a set of related tables (or relations)
- **Relation**: a named, two-dimensional table of data. Each relation consists of a set of named columns and an arbitrary number of unnamed rows
- **Well-Structured Relation**: a relation that contains a minimum amount of redundancy and allows users to insert, modify, and delete the rows without errors or inconsistencies. Also known as table.
- **Properties of Relations:**
  - Entries in cells are simple.
  - Entries in columns are from the same set of values.
  - Each row is unique.
  - The sequence of columns can be interchanged without changing the meaning or use of the relation.
  - The rows may be interchanged or stored in any sequence.

Keys in DBMS

- Key is an attribute or set of attributes which helps you to identify a row(tuple) in a relation(table).
- They allow you to find the relation between two tables.
- Keys help you uniquely identify a row in a table by a combination of one or more columns in that table.
- Key is also helpful for finding unique record or row from the table.

- **Primary Keys**
  - Primary Key
    - An attribute whose value is unique across all occurrences of a relation.
  - All relations have a primary key.
  - This is how rows are ensured to be unique.
  - A primary key may involve a single attribute or be composed of multiple attributes.

- Types of Keys in relational database
  - Super Key
  - Candidate Key
  - Primary Key
  - Alternate Key
  - Foreign Key

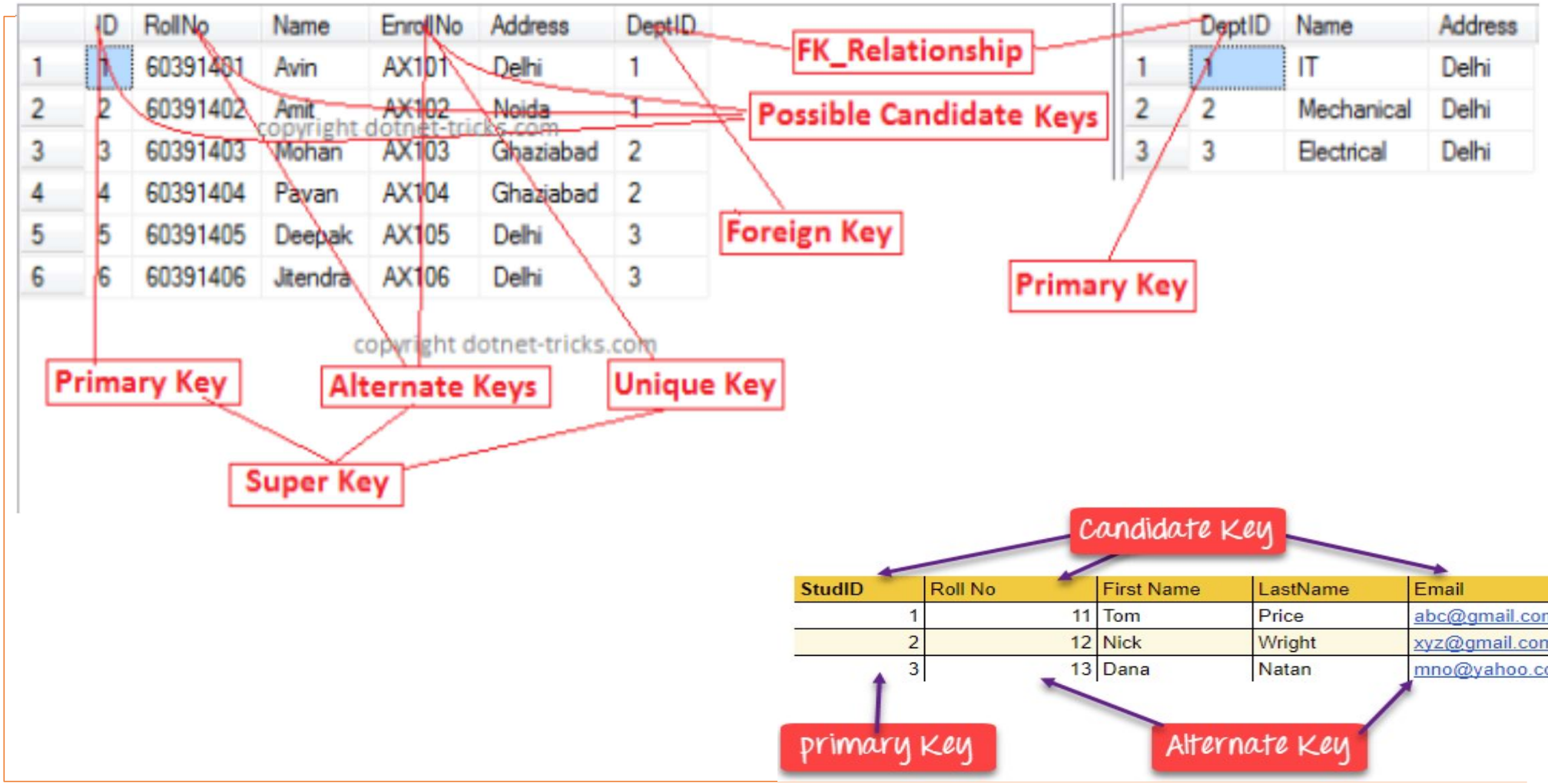
Types of keys in DBMS

- Super Key
  - Super key is a set of one or more than one keys that can be used to identify a record uniquely in a table.
  - **Example :** Primary key, Unique key, Alternate key are subset of Super Keys.
- Candidate Key - minimal superkeys are called Candidate Key
  - A Candidate Key is a set of one or more fields/columns that can identify a record uniquely in a table. There can be multiple Candidate Keys in one table. Each Candidate Key can work as Primary Key.
  - **Example:** In below diagram ID, RollNo and EnrollNo are Candidate Keys since all these three fields can be work as Primary Key.
- Primary Key –
  - Primary key is a set of one or more fields/columns of a table that uniquely identify a record in database table. It can not accept null, duplicate values.
  - *Unique+Not Null*
  - *In a one table only one primary key can exist (i.e no more than one primary key in a table)*

Keys in DBMS

- Foreign Key
  - Foreign Key is a field in database table that is Primary key in another table.
  - It can accept multiple null, duplicate values.
  - **Example**: We can have a DeptID column in the Employee table which is pointing to DeptID column in a department table where it a primary key.
- Alternate key
  - An Alternate key is a key that can be work as a primary key. Basically it is a candidate key that currently is not primary key.
  - **Example**: In below diagram RollNo and EnrollNo becomes Alternate Keys when we define ID as Primary Key.
- Composite/Compound Key
  - Composite Key is a combination of more than one fields/columns of a table. It can be a Candidate key, Primary key.

Keys in DBMS

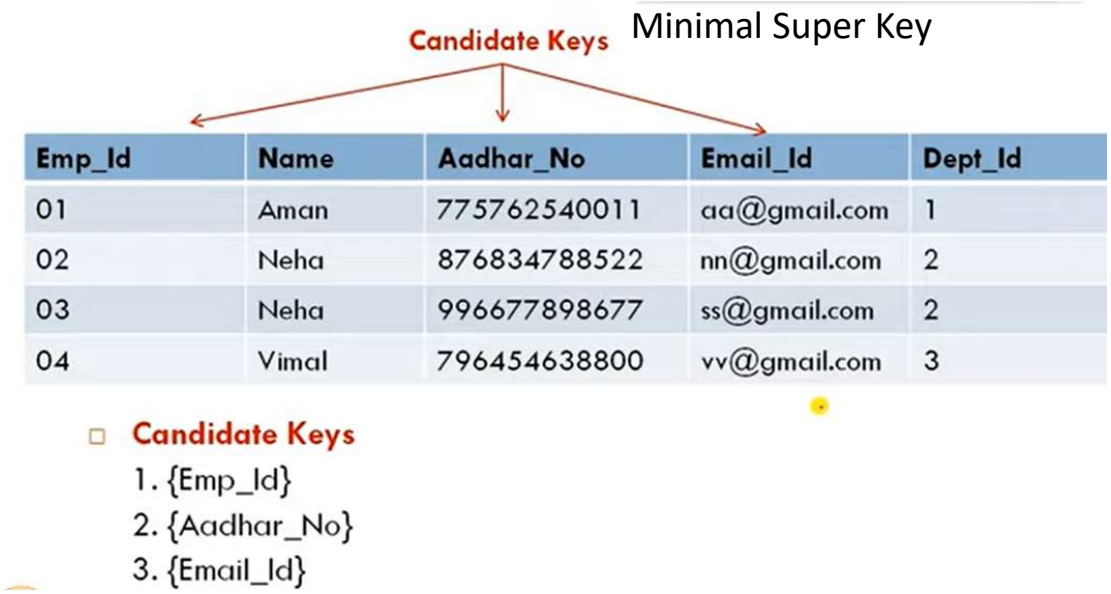

ID
RollNo
Name
EnrollNo
Address
DeptID
1 1 60391401 Avin AX101 Delhi 1
2 2 60391402 Amit AX102 Noida 1
3 3 60391403 Mohan AX103 Ghaziabad 2
4 4 60391404 Pavan AX104 Ghaziabad 2
5 5 60391405 Deepak AX105 Delhi 3
6 6 60391406 Jitendra AX106 Delhi 3
DeptID
Name
Address
1 1 IT Delhi
2 2 Mechanical Delhi
3 3 Electrical Delhi
FK_Relationship
Possible Candidate Keys
Foreign Key
Primary Key
copyright dotnet-tricks.com
copyright dotnet-tricks.com
Primary Key
Alternate Keys
Unique Key
Super Key

Candidate Key
StudID
Roll No
First Name
LastName
Email
1 11 Tom Price abc@gmail.com
2 12 Nick Wright xyz@gmail.com
3 13 Dana Natan mno@yahoo.co
primary Key
Alternate Key

Emp_Id	Name	Aadhar_No	Email_Id	Dept_Id
01	Aman	775762540011	aa@gmail.com	1
02	Neha	876834788522	nn@gmail.com	2
03	Neha	996677898677	ss@gmail.com	2
04	Vimal	796454638800	vv@gmail.com	3

Super Keys :

1. {Emp_ID}
2. {Adhar_no}
3. {Email_id}
4. {Emp_id, Adhar_no}
5. {Adhar_card, Email_id}
6. {Emp_id, Email_id}
7. {Emp_id,Adhar_No, Email_id}
8. {Emp_id, Name}
9. {Emp_id, Dept_id
10. {Emp_id,Name,Adhar_no,Email_id,Dept}
11. Etc ,……



Emp_Id	Name	Aadhar_No	Email_Id	Dept_Id
01	Aman	775762540011	aa@gmail.com	1
02	Neha	876834788522	nn@gmail.com	2
03	Neha	996677898677	ss@gmail.com	2
04	Vimal	796454638800	vv@gmail.com	3

Primary Key:

1. {Emp_ID}

Alternate Key Key:

1. {Adhar_no}
2. {Email_id}

Alternate Key=Candidate Key- Primary Key

Well-Structured Relation and Poorly Structured Relation

Figure 10-5 EMPLOYEE1 relation with sample data

EMPLOYEE1

Emp_ID	Name	Dept	Salary
100	Margaret Simpson	Marketing	42,000
140	Allen Beeton	Accounting	39,000
110	Chris Lucero	Info Systems	41,500
190	Lorenzo Davis	Finance	38,000
150	Susan Martin	Marketing	38,500

No redundancy, and data pertains to a single entity, an employee so is well Structured

Figure 10-6 Relation with redundancy

EMPLOYEE2

Emp_ID	Name	Dept	Salary	Course	Date_Completed
100	Margaret Simpson	Marketing	42,000	SPSS	6/19/2005
100	Margaret Simpson	Marketing	42,000	Surveys	10/7/2005
140	Alan Beeton	Accounting	39,000	Tax Acc	12/8/2005
110	Chris Lucero	Info Systems	41,500	SPSS	1/22/2005
110	Chris Lucero	Info Systems	41,500	C++	4/22/2005
190	Lorenzo Davis	Finance	38,000	Investments	5/7/2005
150	Susan Martin	Marketing	38,500	SPSS	6/19/2005
150	Susan Martin	Marketing	38,500	TQM	8/12/2005

Redundancies, because data pertains to a two entities, employees and the courses they take so is poorly structured

Normalization

- The process of converting complex data structures into simple, stable data structures
- Normalization is a technique of organizing the data in the database.
- Normalization is a systematic approach of decomposing tables to eliminate data redundancy(repetition) and undesirable characteristics like Insertion, Update and Deletion Anomalies.
- It is a multi-step process that puts data into tabular form, removing duplicated data from the relation tables.
- Normalization is used for mainly two purposes,
  - Eliminating redundant(useless) data.
  - Ensuring data dependencies make sense i.e data is logically stored.

Problems Without Normalization

- If a table is not properly normalized and have data redundancy then it will not only eat up **extra memory** space but will also make it **difficult to handle and update the database**, without facing data loss. Insertion, Updation and Deletion Anomalies are very frequent if database is not normalized. To understand these anomalies let us take an example of a **Student** table.

rollno	name	branch	hod	office_tel
401	Akon	CSE	Mr. X	53337
402	Bkon	CSE	Mr. X	53337
403	Ckon	CSE	Mr. X	53337
404	Dkon	CSE	Mr. X	53337

- In the table above ,we have data of four computer sci.students. As we can see, data for the fields *branch, hod* and *office_tel* is repeated for the students who are in the same branch in the college, this is the **data redundancy.**
- **Insertion Anomaly**
  - Suppose for a new admission, until and unless a student opts for a branch, data of the student cannot be inserted, or else we will have to set the branch information as **NULL**.
  - Also, if we have to insert data of 100 students of same branch, then the branch information will be repeated for all those 100 students.
  - These scenarios are nothing but **Insertion anomalies**.

- **Updation Anomaly**
  - What if Mr. X leaves the college? or is no longer the HOD of computer science department? In that case all the student records will have to be updated, and if by mistake we miss any record, it will lead to data inconsistency. This is Updation anomaly.
- **Deletion Anomaly**
  - In our **Student** table, two different informations are kept together, Student information and Branch information. Hence, at the end of the academic year, if student records are deleted, we will also lose the branch information. This is Deletion anomaly.
- **Normalization Rule**
- Normalization rules are divided into the following normal forms:
  - First Normal Form
  - Second Normal Form
  - Third Normal Form
  - BCNF
  - Fourth Normal Form
- **First Normal Form (1NF)**
- For a table to be in the First Normal Form, it should follow the following 4 rules:
  - It should only have single(atomic) valued attributes/columns.
  - Values stored in a column should be of the same domain
  - All the columns in a table should have unique names.
  - And the order in which data is stored, does not matter.

- **Second Normal Form (2NF)**
- For a table to be in the Second Normal Form,
  - It should be in the First Normal form.
  - And, it should not have Partial Dependency.
- **Third Normal Form (3NF)**
- A table is said to be in the Third Normal Form when,
  - It is in the Second Normal form.
  - And, it doesn't have Transitive Dependency.
- **Boyce and Codd Normal Form (BCNF)**
- **Boyce and Codd Normal Form** is a higher version of the Third Normal form. This form deals with certain type of anomaly that is not handled by 3NF. A 3NF table which does not have multiple overlapping candidate keys is said to be in BCNF. For a table to be in BCNF, following conditions must be satisfied:
  - R must be in 3rd Normal Form
  - and, for each functional dependency (X → Y), X should be a super Key.
- **Fourth Normal Form (4NF)**
- A table is said to be in the Fourth Normal Form when,
  - It is in the Boyce-Codd Normal Form.
  - And, it doesn't have Multi-Valued Dependency.

- **Rules for First Normal Form**
- Rule 1: Single Valued Attributes
- Rule 2: Attribute Domain should not change
- Rule 3: Unique name for Attributes/Columns
- Rule 4: Order doesn't matters
- **Example:**

roll_no	name	subject
101	Akon	OS, CN
103	Ckon	Java
102	Bkon	C, C++

- Convert the above table in 1NF as:

roll_no	name	subject
101	Akon	OS
101	Akon	CN
103	Ckon	Java
102	Bkon	C
102	Bkon	C++

- **Rules for Second Normal Form**
- The table should be in the First Normal Form.
- There should be no Partial Dependency.
- **What is Dependency?**
- Let's take an example of a **Student** table with columns std_id,name,reg_no,branch, and address.

student_id	name	reg_no	branch	address

In this table, student_id is the primary key and will be unique for every row, hence we can use student_id to fetch any row of data from this table.
Even for a case, where student names are same, if we know the student_id we can easily fetch the correct record.

student_id	name	reg_no	branch	address
10	Akon	07-WY	CSE	Kerala
11	Akon	08-WY	IT	Gujarat

Hence we can say a **Primary Key** for a table is the column or a group of columns(composite key) which can uniquely identify each record in the table.We can ask from branch name of student with student_id **10**, and can get it. Similarly, if we ask for name of student with student_id **10** or **11**, we will get it. So all the need is student_id and every other column **depends** on it, or can be fetched using it.
This is **Dependency** and we also call it **Functional Dependency**.

- What is partial dependency?
- Let's create another table for **Subject**, which will have subject_id and subject_name as a field and subject_id as a primary key.

subject_id	subject_name
1	Java
2	C++
3	Php
Table:Subject	

student_id	name	reg_no	branch	address
10	Akon	07-WY	CSE	Kerala
11	Akon	08-WY	IT	Gujarat
Table:Student				

Let's create another table **Score**, to store the **marks** obtained by students in the respective subjects. We will also be saving **name of the teacher** who teaches that subject along with marks.

score_id	student_id	subject_id	marks	teacher
1	10	1	70	Java Teacher
2	10	2	75	C++ Teacher
3	11	1	80	Java Teacher
Table:Score				

In the score table we are saving the **student_id** to know which student's marks are these and **subject_id** to know for which subject the marks are for.

Now ,If we ask to get marks with the student with student id 10,we could not give the answer because we don't know for which subject.And if we have subject_id we would not know for which student to give marks.So a composite key of (student_id+subject_id) is to make a primary key.

- Where is the partial dependency?

- Now if you look at the **Score** table, we have a column names ***teacher*** which is only dependent on the subject, for Java it's Java Teacher and for C++ it's C++ Teacher & so on.
- Here teacher name is only dependent on subjet and hence to the subject_id only but not with the student_id. This is **Partial Dependency**, where an attribute in a table depends on only a part of the primary key and not on the whole key.
- The solution of above partial dependency is done simply by removing the column teacher from the score table and add it to subject table as:
- Now the table Score is in 2NF with no partial dependency as:

score_id	student_id	subject_id	marks
1	10	1	70
2	10	2	75
3	11	1	80
Table :Score			

subject_id	subject_name	teacher
1	Java	Java Teacher
2	C++	C++ Teacher
3	Php	Php Teacher
New table:Subject		

Summary of 2NF:

1.For a table to be in the Second Normal form, it should be in the First Normal form and it should not have Partial Dependency.

2.Partial Dependency exists, when for a composite primary key, any attribute in the table depends only on a part of the primary key and not on the complete primary key.

3.To remove Partial dependency, we can divide the table, remove the attribute which is causing partial dependency, and move it to some other table where it fits in well.

• Requirements for Third Normal Form

- For a table to be in the third normal form,
  - It should be in the Second Normal form.
  - And it should not have Transitive Dependency.
- Let us take the Score table, we need to store some more information, which is the exam name and total marks, so let's add 2 more columns to the Score table.

score_id	student_id	subject_id	marks	exam_name	total_marks
NEW TABLE:SCORE					

- Since exam_name depends on both student and subject so the primary key will be:student_id+subject_id
- The next column total_marks only depends on exam_name as with exam type the total score changes. For example, practicals are of less marks while theory exams are of more marks.
- But exam_name is just another column in the score table. It is not a primary key or even a part of the primary key, and total_marks depends on it.
- This situation is Transitive Dependency. When a non-prime attribute depends on other non-prime attributes rather than depending upon the prime attributes or primary key.
- In order to solve this issue take out the columns exam_name and total_marks from score table and put them in exam table and make exam_id as a primary key.Here the exam_id is the **foreign key** for the table score.

score_id	student_id	subject_id	marks	exam_id
Table:Score				

exam_id	exam_name	total_marks
1	Workshop	200
2	Mains	70
Table:Exam		

- Rules for BCNF:

- For a table to satisfy the Boyce-Codd Normal Form, it should satisfy the following two conditions:
  - It should be in the **Third Normal Form**.
  - And, for any dependency A → B, A should be a **super key**. This means that for a dependency A → B, A cannot be a **non-prime attribute**, if B is a **prime attribute**.
- Example:
- Let us select a college enrollment table with columns:student_id,subject,Professor as:

student_id	subject	professor
101	Java	P.Java
101	C++	P.Cpp
102	Java	P.Java2
103	C#	P.Chash
104	Java	P.Java
TABLE:College_Enrollment		

In the table above:
•One student can enrol for multiple subjects. For example, student with **student_id** 101, has opted for subjects - Java & C++
•For each subject, a professor is assigned to the student.
•And, there can be multiple professors teaching one subject like we have for Java.
•Here the primary key will be student_id and subject to find all the columns in the table.

- One more important point to note here is, one professor teaches only one subject, but one subject may have two different professors. Hence, there is a dependency between subject and professor because subject depends on professor.
- Here the professor is non prime attributes and subject is prime attribute.BCNF does not allow this situation.That is prime attribute(subject) can not be depend on non prime attribute(Professor).
- To make this relation(table) satisfy BCNF, we will decompose this table into two tables, **student** table and **professor** table.
- Student table contain student_id and Professor_id and Professor table contain professor_id,professor and subjects.

Student Table

student_id	p_id
101	1
101	2

Professor Table

p_id	professor	subject
1	P.Java	Java
2	P.Cpp	C++

In the picture below, we have tried to explain BCNF in terms of relation:

Consider the following relationship : **R (A,B,C,D)**

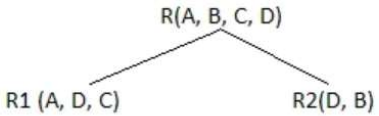
and following dependencies :

A -> BCD
BC -> AD
D -> B

Above relationship is already in 3rd NF. Keys are **A** and **BC.**

Hence, in the functional dependency, **A -> BCD**, A is the super key.
in second relation, **BC -> AD**, BC is also a key.
but in, **D -> B**, D is not a key.

Hence we can break our relationship R into two relationships **R1** and **R2**.



Breaking, table into two tables, one with A, D and C while the other with D and B.

• Rules for 4th Normal Form

- For a table to satisfy the Fourth Normal Form, it should satisfy the following two conditions:
  - It should be in the **Boyce-Codd Normal Form**.
  - And, the table should not have any **Multi-valued Dependency**.
- A table is said to have multi-valued dependency, if the following conditions are true,
  - For a dependency A → B, if for a single value of A, multiple value of B exists, then the table may have multi-valued dependency.
  - Also, a table should have at-least 3 columns for it to have a multi-valued dependency.
  - For a relation R(A,B,C), if there is a multi-valued dependency between, A and B, then B and C should be independent of each other.
  - If all these conditions are true for any relation(table), it is said to have multi-valued dependency.
- Example:

s_id	course	hobby
1	Science	Cricket
1	Maths	Hockey
2	C#	Cricket
2	Php	Hockey
Table:College_Enrollment		

- Here the Student with s_id **1** has opted for two courses, **Science** and **Maths**, and has two hobbies, **Cricket** and **Hockey**.
- Well the two records for student with s_id **1** will give rise to two more records, as shown below, because for one student, two hobbies exists, hence along with both the courses, these hobbies should be specified.There is no relationships between course and hobby. So there is multi-value dependency, which leads to un-necessary repetition of data and other anomalies as well.

s_id	course	hobby
1	Science	Cricket
1	Maths	Hockey
1	Science	Hockey
1	Maths	Cricket
Table: College Enrollment		

- **How to satisfy 4th Normal Form?**
- We can just decompose the table in two different tables as:

Table:CourseOpted

s_id	course
1	Science
1	Maths
2	C#
2	Php

Hobbies Table,

s_id	hobby
1	Cricket
1	Hockey
2	Cricket
2	Hockey

- Now this relation satisfies the fourth normal form.
- A table can also have functional dependency along with multi-valued dependency. In that case, the **functionally dependent** columns are moved in a **separate table** and the **multi-valued dependent** columns are moved to **separate tables.**

Normalize the below relations upto 3NF

Full Names	Physical Address	Movies Rented	Salutation
Janet Jones	First Street Plot No 4	Pirates of the Caribbean, Clash of the Titans	Ms.
Robert Phil	3rd Street 34	Forgetting Sarah Marshal, Daddy's Little Girls	Mr.
Robert Phil	5th Avenue	Clash of the Titans	Mr.

Full Names	Physical Address	Movies Rented	Salutation
Janet Jones	First Street Plot No 4	Pirates of the Caribbean	Ms.
Janet Jones	First Street Plot No 4	Clash of the Titans	Ms.
Robert Phil	3rd Street 34	Forgetting Sarah Marshal	Mr.
Robert Phil	3rd Street 34	Daddy's Little Girls	Mr.
Robert Phil	5th Avenue	Clash of the Titans	Mr.

1NF

Membership ID	Full Names	Physical Address	Salutation
1	Janet Jones	First Street Plot No 4	Ms.
2	Robert Phil	3rd Street 34	Mr.
3	Robert Phil	5th Avenue	Mr.

Membership ID	Movies Rented
1	Pirates of the Caribbean
1	Clash of the Titans
2	Forgetting Sarah Marshal
2	Daddy's Little Girls
3	Clash of the Titans

2NF

Membership ID	Full Names	Physical Address	Salutation
1	Janet Jones	First Street Plot No 4	Ms.
2	Robert Phil	3rd Street 34	Mr.
3	Robert Phil	5th Avenue	Mr.

Change in Name

May Change Salutation

Membership ID	Full Names	Physical Address	Salutation ID
1	Janet Jones	First Street Plot No 4	2
2	Robert Phil	3rd Street 34	1
3	Robert Phil	5th Avenue	1

Membership ID	Movies Rented
1	Pirates of the Caribbean
1	Clash of the Titans
2	Forgetting Sarah Marshal
2	Daddy's Little Girls
3	Clash of the Titans

Salutation ID	Salutation
1	Mr.
2	Ms.
3	Mrs.
4	Dr.

3NF

Normal Form	Description
1NF	A relation is in 1NF if it contains an atomic value. **(no multivalued attribute)**
2NF	A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key. **(No partial Dependency)**
3NF	A relation will be in 3NF if it is in 2NF and **no transition dependency** exists.
BCNF	3 NF+ LHS must be Candidate Key or Super Key
4NF	A relation will be in 4NF if it is in Boyce Codd normal form and has **no multi-valued dependency.** E.G A person can have multiple phone no and multiple email , Then we should make two table one contains Name and email , other Name with phone .
5NF	A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless. **(lossless decomposition) – No lossy decomposition** **Also called Project-Join Normal Form (PJNC)**
DKNF	**The constraints are verified by the domain and key constraints** Domain-Key Normal Form is the highest form of Normalization.

Designing Database relations example

Simple example of logical data modeling (a) Highest-volume customer query screen

(b) Backlog summary report

(a)

HIGHEST-VOLUME CUSTOMER

ENTER PRODUCT ID.:	M128
START DATE:	11/01/2017
END DATE:	12/31/2017
CUSTOMER ID.:	1256
NAME:	Commonwealth Builder
VOLUME:	30

This inquiry screen shows the customer with the largest volume of total sales for a specified product an indicated time period.

Relations:

CUSTOMER(Customer_ID, Name)
ORDER(Order_Number, Customer_ID, Order_Date)
PRODUCT(Product_ID)
LINE ITEM(Order_Number, Product_ID, Order_Quantity)

(b)

PAGE 1

BACKLOG SUMMARY REPORT
11/30/2017

PRODUCT ID	BACKLOG QUANTITY
B381	0
B975	0
B985	6
E125	30
⋮	
M128	2
⋮	

This report shows the unit volume of each product that has been ordered less that amount shipped through the specified date.

Relations:

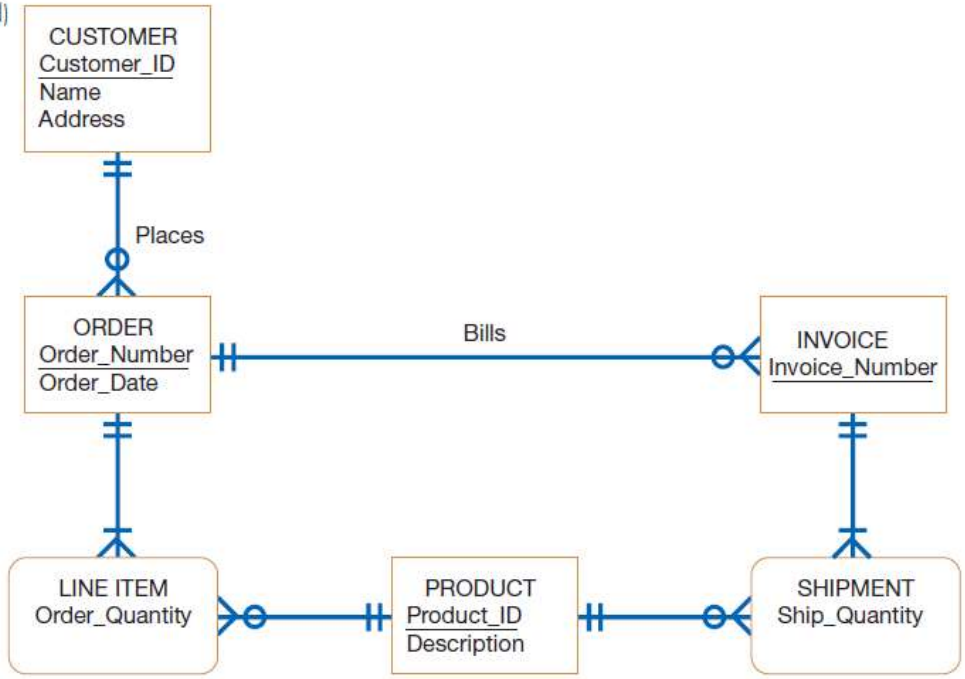
PRODUCT(Product_ID)
LINE ITEM(Product_ID, Order_Number, Order_Quantity)
ORDER(Order_Number, Order_Date)
SHIPMENT(Product_ID, Invoice_Number, Ship_Quantity)
INVOICE(Invoice_Number, Invoice_Date, Order_Number)

(c) CUSTOMER(Customer_ID, Name)
PRODUCT(Product_ID)
INVOICE(Invoice_Number, Invoice_Date, Order_Number)
ORDER(Order_Number, Customer_ID, Order_Date)
LINE ITEM(Order_Number, Product_ID, Order_Quantity)
SHIPMENT(Product_ID, Invoice_Number, Ship_Quantity)

(c) Integrated set of relations

FIGURE 9-3 (continued)
(d) Conceptual data model and transformed relations

(d)



Relations:

CUSTOMER(Customer_ID,Name,Address)
PRODUCT(Product_ID,Description)
ORDER(Order_Number,Customer_ID,Order_Date)
LINE ITEM(Order_Number,Product_ID,Order_Quantity)
INVOICE(Invoice_Number,Order_Number)
SHIPMENT(Invoice_Number,Product_ID,Ship_Quantity)

(e) Final set of normalized relations

(e) CUSTOMER(Customer_ID,Name,Address)
PRODUCT(Product_ID,Description)
ORDER(Order_Number,Customer_ID,Order_Date)
LINE ITEM(Order_Number,Product_ID,Order_Quantity)
INVOICE(Invoice_Number,Order_Number,Invoice_Date)
SHIPMENT(Invoice_Number,Product_ID,Ship_Quantity)

Transforming E-R Diagrams into Relations

- It is useful to transform the conceptual data model into a set of normalized relations
- Steps
  - Represent entities
  - Represent relationships
  - Normalize the relations
  - Merge the relations
- **Representing Entities**
  - Each regular entity is transformed into a relation.
  - The identifier of the entity type becomes the primary key of the corresponding relation.
  - The primary key must satisfy the following two conditions.
    a. The value of the key must uniquely identify every row in the relation.
    b. The key should be non redundant.

Figure 10-10a Transforming an entity type to a relation - E-R Diagram

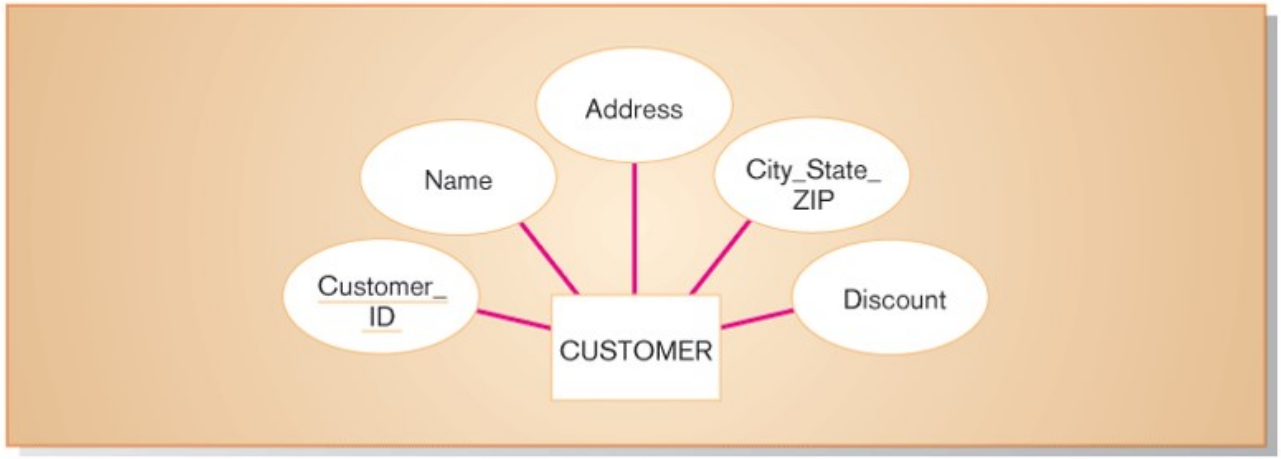


Figure 10-10b Transforming an entity type to a relation - Relation

CUSTOMER

Customer_ID	Name	Address	City_State_ZIP	Discount
1273	Contemporary Designs	123 Oak St.	Austin, TX 28384	5%
6390	Casual Corner	18 Hoosier Dr.	Bloomington, IN 45821	3%

Represent Relationships

- Binary 1:N Relationships
  - Add the primary key attribute (or attributes) of the entity on the one side of the relationship as a foreign key in the relation on the right side.
  - The one side *migrates* to the many side.
- Binary or Unary 1:1
  - Three possible options
    a. Add the primary key of A as a foreign key of B.
    b. Add the primary key of B as a foreign key of A.
    c. Both of the above.

Figure 10-11a Representing a 1:N relationship - E-R Diagram

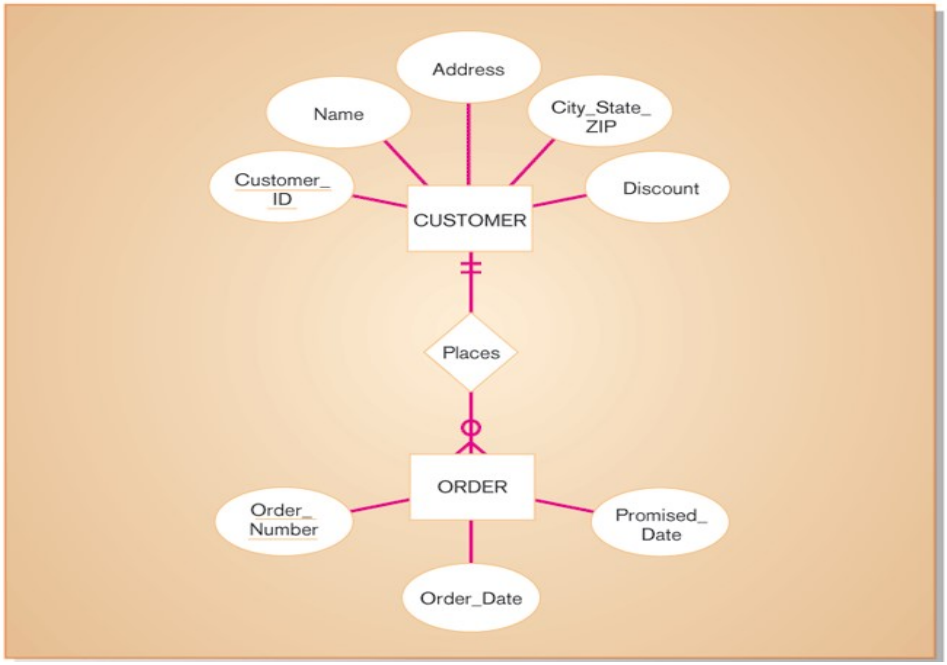


Figure 10-11b Representing a 1:N relationship - Relations

CUSTOMER

Customer_ID	Name	Address	City_State_ZIP	Discount
1273	Contemporary Designs	123 Oak St.	Austin, TX 28384	5%
6390	Casual Corner	18 Hoosier Dr.	Bloomington, IN 45821	3%

ORDER

Order_Number	Order_Date	Promised_Date	Customer_ID
57194	3/15/0X	3/28/0X	6390
63725	3/17/0X	4/01/0X	1273
80149	3/14/0X	3/24/0X	6390

- Binary and Higher M:N relationships
- Create another relation and include primary keys of all relations as primary key of new relation.

Figure 10-12a Representing an M:N relationship - E-R Diagram

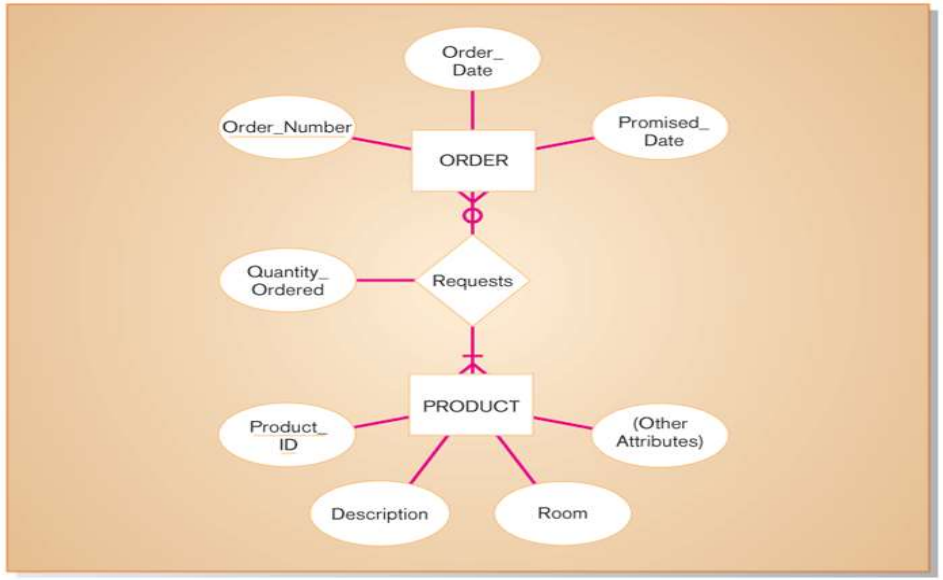


Figure 10-12b Representing an M:N relationship - Relations

ORDER

Order_Number	Order_Date	Promised_Date
61384	2/17/2005	3/01/2005
62009	2/13/2005	2/27/2005
62807	2/15/2005	3/01/2005

ORDER LINE

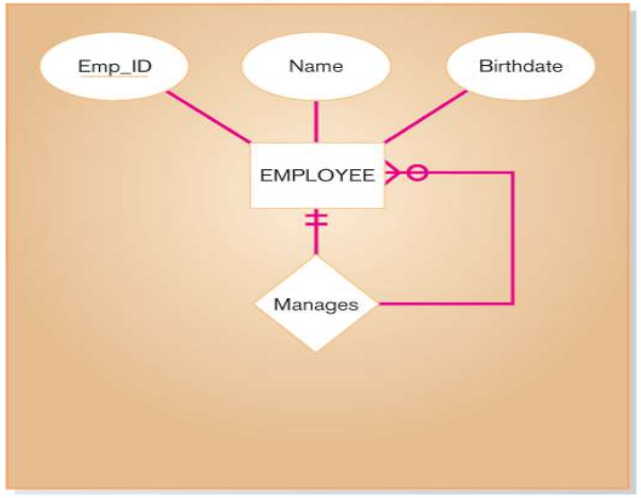
Order_Number	Product_ID	Quantity_ Ordered
61384	M128	2
61384	A261	1

PRODUCT

Product_ID	Description	Room	(Other Attributes)
M128	Bookcase	Study	–
A261	Wall unit	Family	–
R149	Cabinet	Study	–

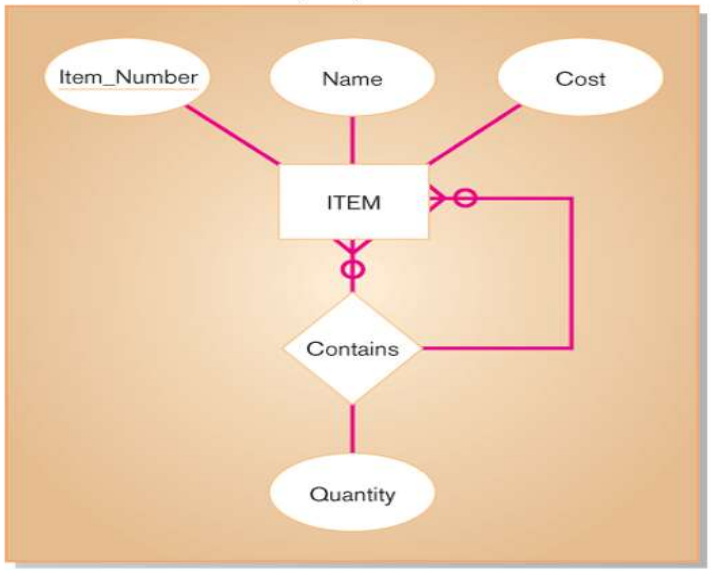
- Unary 1:N Relationships
  - Relationship between instances of a single entity type
  - Utilize a recursive foreign key
    - A foreign key in a relation that references the primary key values of that same relation.
- Unary M:N Relationships
  - Create a separate relation.
  - Primary key of new relation is a composite of two attributes that both take their values from the same primary key.

Figure 10-13a Two unary relations - EMPLOYEE with Manages relationship (1:N)



EMPLOYEE(Emp_ID, Name, Birthdate, Manager_ID)

Figure 10-13b Two unary relations - Bill-of-materials structure (M:N)



ITEM(Item_Number, Name, Cost)
ITEMCOMPONENT(Item_Number, Component_Number, Quatity)

Merging Relations (View Integration)

- Purpose is to remove redundant relations
- View Integration Problems
  - Synonyms
    - Two different names used for the same attribute
    - When merging, get agreement from users on a single, standard name
    - Example:
      - **STUDENT1 (Student ID, Name)**
      - **STUDENT2 (Matriculation Number,Name,Address)**
      - **STUDENT (SSN,Name,Address)**
  - Homonyms
    - A single attribute name that is used for two or more different attributes
    - Resolved by creating a new name
    - Example:
      - **STUDENT1 (Student ID, Name,Address)**
      - **STUDENT2 (Student ID,Name,Phone_Number,Address)**
      - **STUDENT (Student ID,Name,Phone_Number,Campus_Address,Permanent_Address)**
  - Dependencies between nonkeys
    - Dependencies may be created as a result of view integration
    - In order to resolve, the new relation must be normalized
    - Example:
      - **STUDENT1 (Student ID,Major)**
      - **STUDENT2 (Student ID,Adviser)**
      - **STUDENT (Student ID,Major,Adviser)**
    - **Here Major->Advisor so, the table can be formed as:**
      - **STUDENT(Student ID,Major)**
      - **MAJORADVISOR(Major,Advisor)**

Physical File and Database Design

- The following information is required:
  - Normalized relations, including volume estimates
  - Definitions of each attribute
  - Descriptions of where and when data are used, entered, retrieved, deleted, and updated (including frequencies)
  - Expectations or requirements for response time and data integrity
  - Descriptions of the technologies used for implementing the files and database

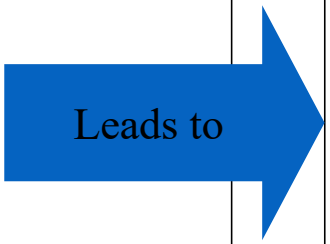
✖Purpose–translate the logical description of data into the *technical specifications* for storing and retrieving data

✖Goal–create a design for storing data that will provide *adequate performance* and insure *database integrity*, *security*, and *recoverability*

Physical Design Process

Inputs

- Normalized relations
- Volume estimates
- Attribute definitions
- Response time expectations
- Data security needs
- Backup/recovery needs
- Integrity expectations
- DBMS technology used



Decisions

- Attribute data types
- Physical record descriptions (doesn't always match logical design)
- File organizations
- Indexes and database architectures
- Query optimization

Designing Fields

- Field
  - Smallest unit of named application data recognized by system software
  - Attributes from relations will be represented as fields
- Field design
  - Choosing data type
  - Coding, compression, encryption
  - Controlling data integrity
- Data Type
  - A coding scheme recognized by system software for representing organizational data
- Choosing data types
  - Four objectives
    - Minimize storage space
    - Represent all possible values of the field
    - Improve data integrity of the field
    - Support all data manipulations desired on the field
  - Calculated fields
    - A field that can be derived from other database fields

Choosing Data Types

TABLE 5-1 Commonly Used Data Types in Oracle 11g

Data Type	Description
VARCHAR2	Variable-length character data with a maximum length of 4,000 characters; you must enter a maximum field length [e.g., VARCHAR2(30) specifies a field with a maximum length of 30 characters]. A string that is shorter than the maximum will consume only the required space.
CHAR	Fixed-length character data with a maximum length of 2,000 characters; default length is 1 character (e.g., CHAR(5) specifies a field with a fixed length of 5 characters, capable of holding a value from 0 to 5 characters long).
CLOB	Character large object, capable of storing up to (4 gigabytes – 1) * (database block size) of one variable-length character data field (e.g., to hold a medical instruction or a customer comment).
NUMBER	Positive or negative number in the range 10^{-130} to 10^{126}; can specify the precision (total number of digits to the left and right of the decimal point) and the scale (the number of digits to the right of the decimal point). For example, NUMBER(5) specifies an integer field with a maximum of 5 digits, and NUMBER(5,2) specifies a field with no more than 5 digits and exactly 2 digits to the right of the decimal point.
DATE	Any date from January 1, 4712 B.C., to December 31, 9999 A.D.; DATE stores the century, year, month, day, hour, minute, and second.
BLOB	Binary large object, capable of storing up to (4 gigabytes – 1) * (database block size) of binary data (e.g., a photograph or sound clip).

Methods of Controlling Data Integrity/Field Data Integrity

- Default Value
  - A value a field will assume unless an explicit value is entered for that field
- Range Control
  - Limits range of values that can be entered into field
- Referential Integrity
  - An integrity constraint specifying that the value (or existence) of an attribute in one relation depends on the value (or existence) of the same attribute in another relation
- Null Value
  - A special field value, distinct from 0, blank, or any other value, that indicates that the value for the field is missing or otherwise unknown

Figure 10-17a Examples of referential integrity field controls - Referential integrity between relations

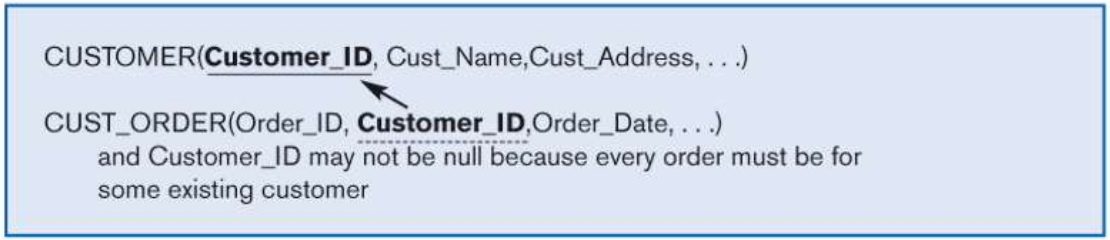
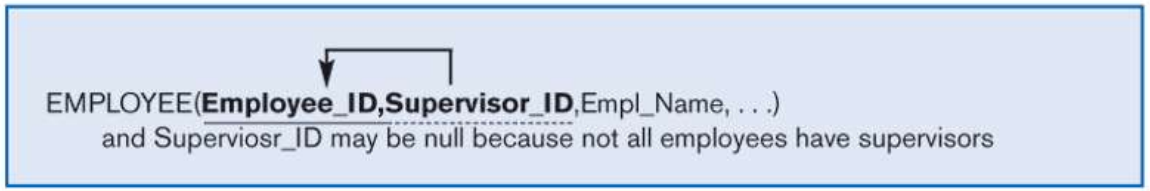


Figure 10-17b Examples of referential integrity field controls - Referential integrity within a relation



Handling Missing Data

- ✖Substitute an estimate of the missing value (e.g., using a formula)
- ✖Construct a report listing missing values
- ✖In programs, ignore missing data unless the value is significant (sensitivity testing)

Triggers can be used to perform these operations

Denormalization

- The process of splitting or combining normalized relations into physical tables based on affinity of use of rows and fields
- Benefits:
  - Can improve performance (speed) by reducing number of table lookups (i.e. *reduce number of necessary join queries*)
- Costs (due to data duplication)
  - Wasted storage space
  - Data integrity/consistency threats

- Partitioning
  - Capability to split a table into separate sections
  - Oracle 9i implements three types
    - Range
    - Hash
    - Composite
- Optimizes certain operations at the expense of others
- **When to Denormalize?**
  - Three common situations where denormalization may be used
    1. Two entities with a one-to-one relationship
    2. A many-to-many relationship with nonkey attributes
    3. Reference data

Figure :A possible denormalization situation: two entities with one-to-one relationship

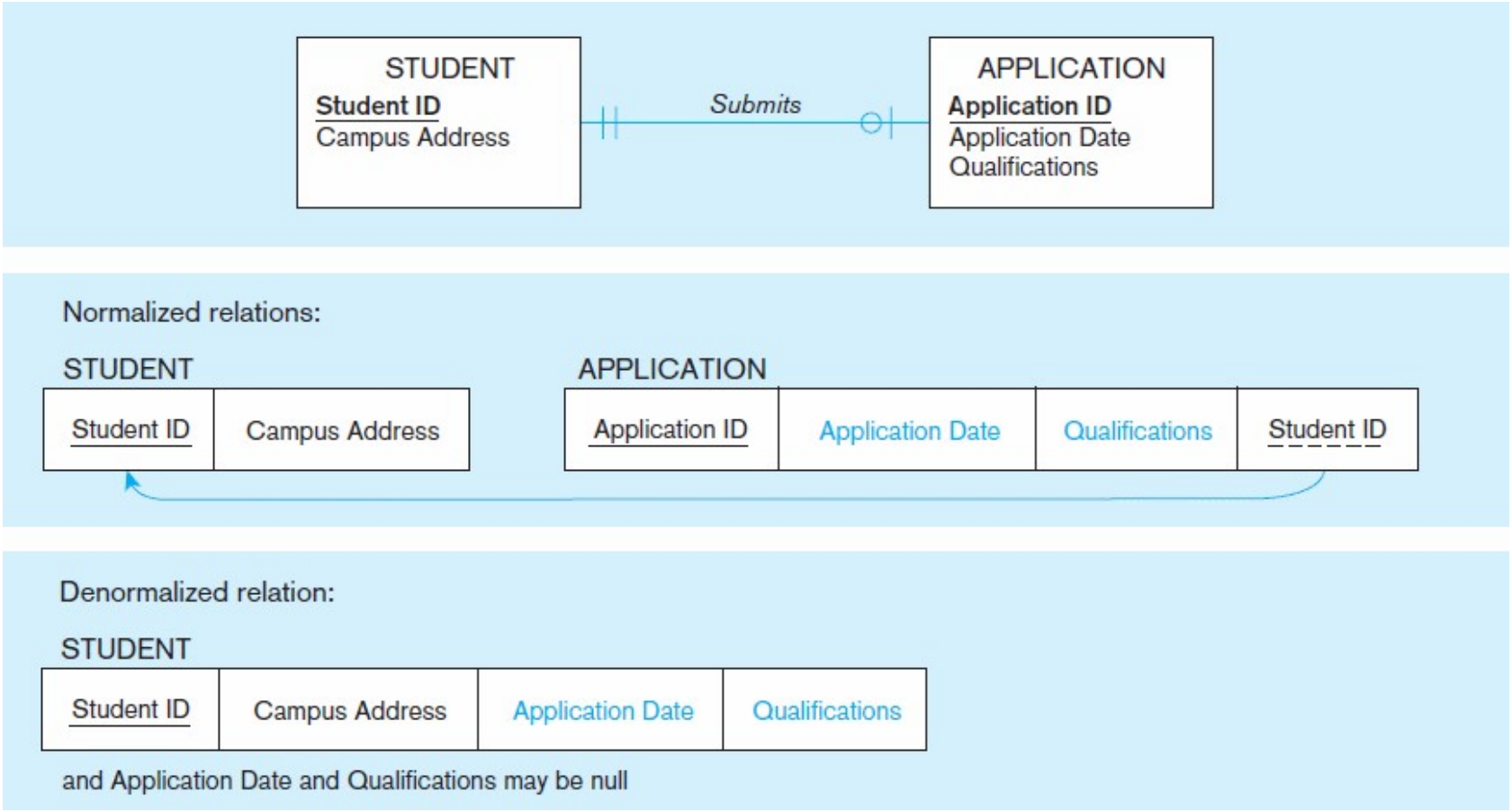


Figure : A possible denormalization situation: a many-to-many relationship with nonkey attributes

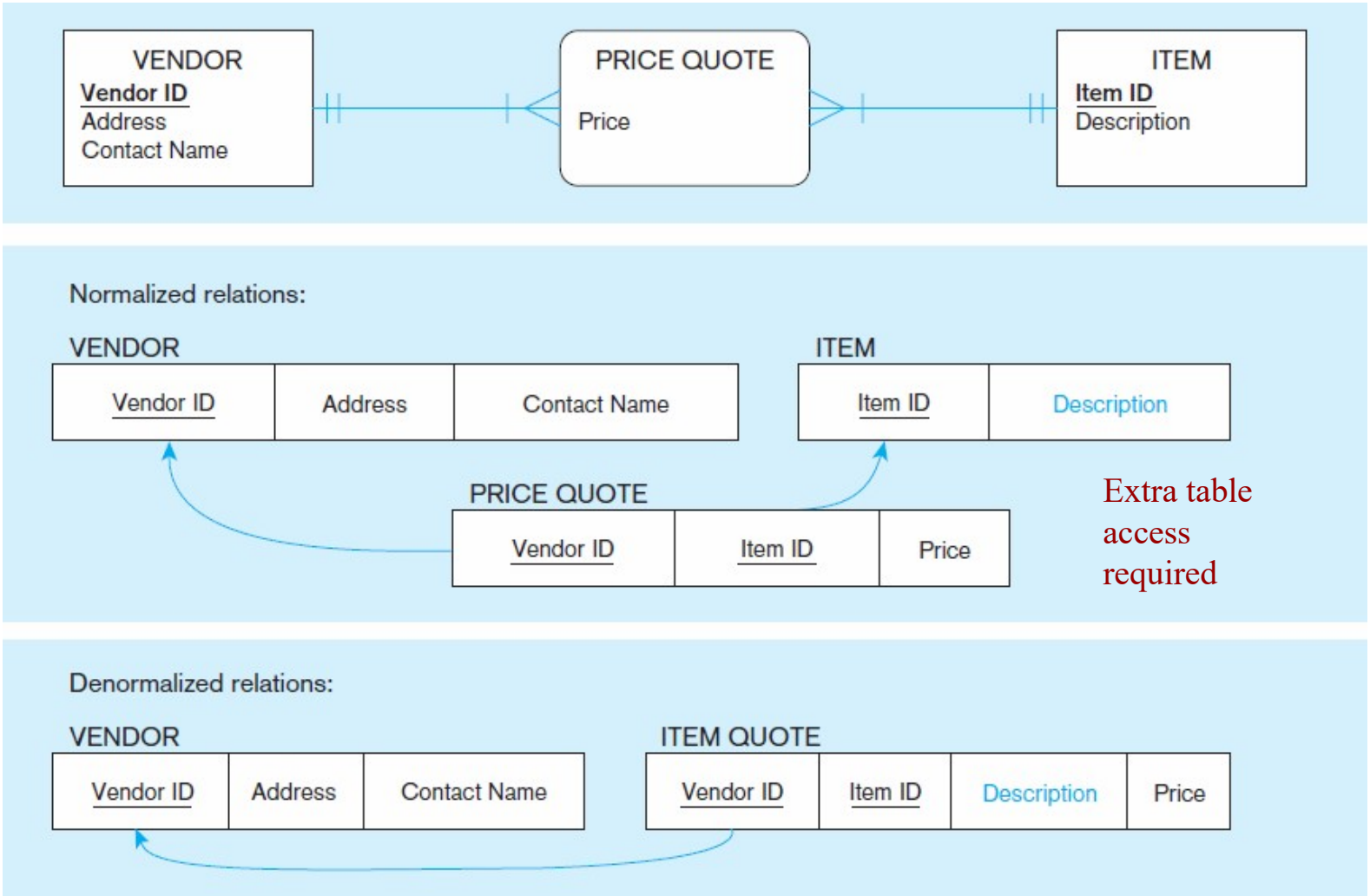


Figure :
A possible denormalization situation: reference data

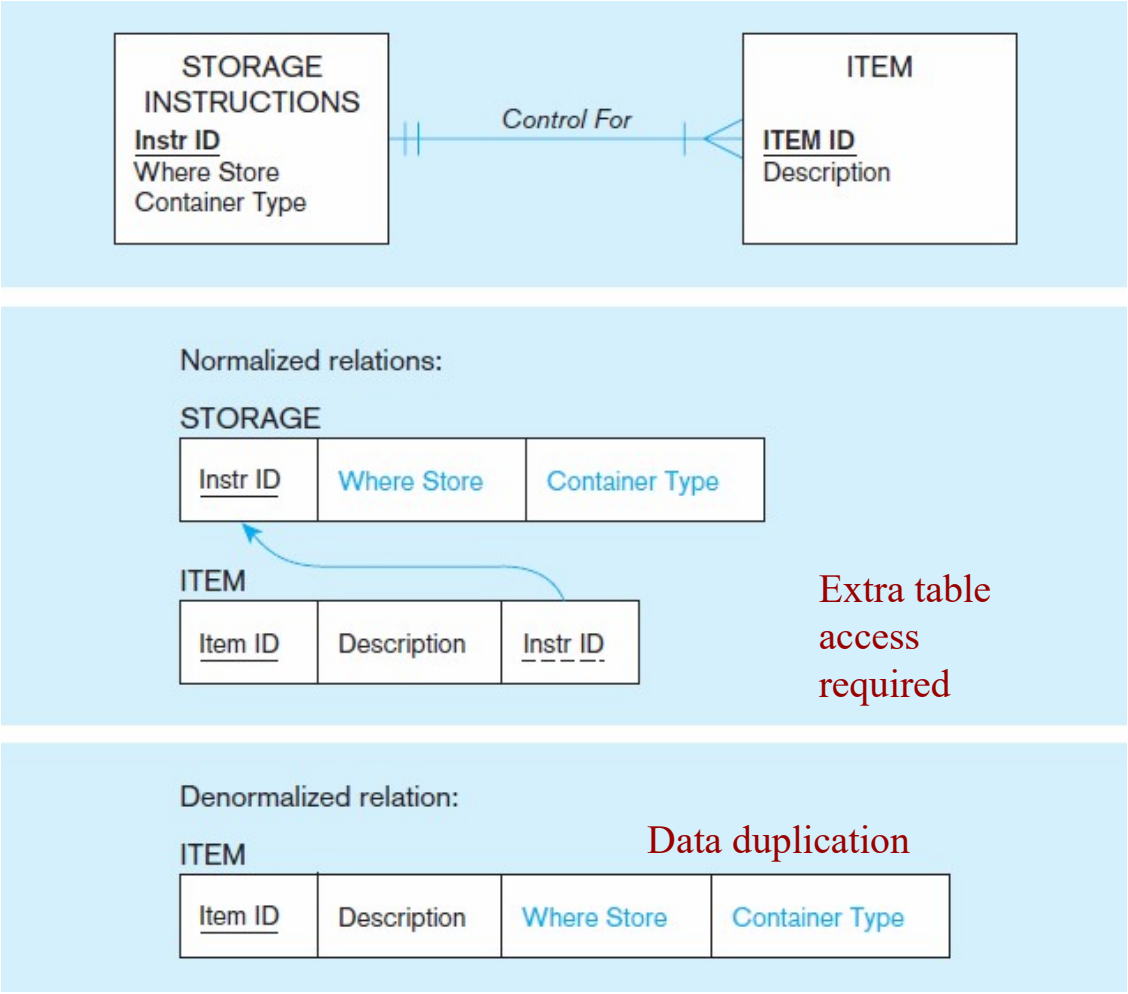
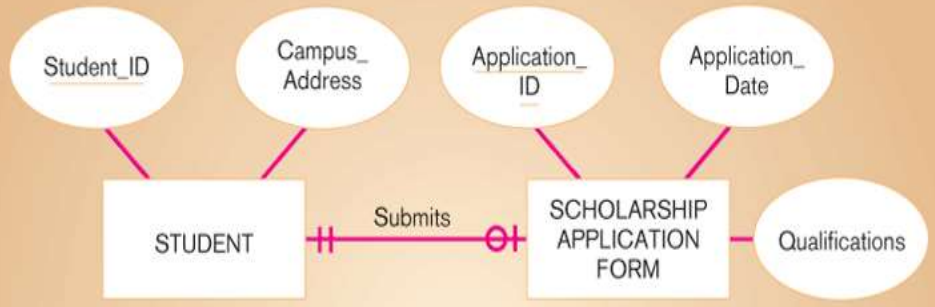


Figure 10-19a Possible denormalization situations - Two entities with a one-to-one relationship



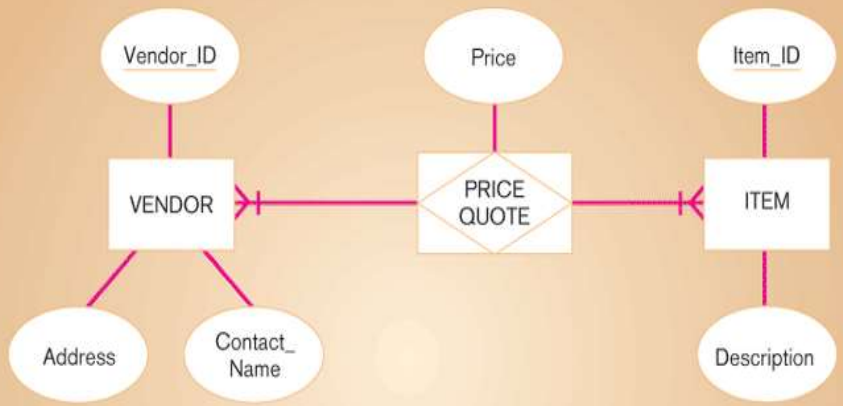
Normalized relations:
STUDENT(Student_ID,Campus_Address,Application_ID)
APPLICATION(Application_ID,Application_Date,Qualifications,Student_ID)

Denormalized relation:
STUDENT(Student_ID,Campus_Address,Application_Date,Qualifications)

and Application_Date and Qualifications may be null

(**Note:** We assume Application_ID is not necessary when all fields are stored in one record, but this field can be included if it is required application data.)

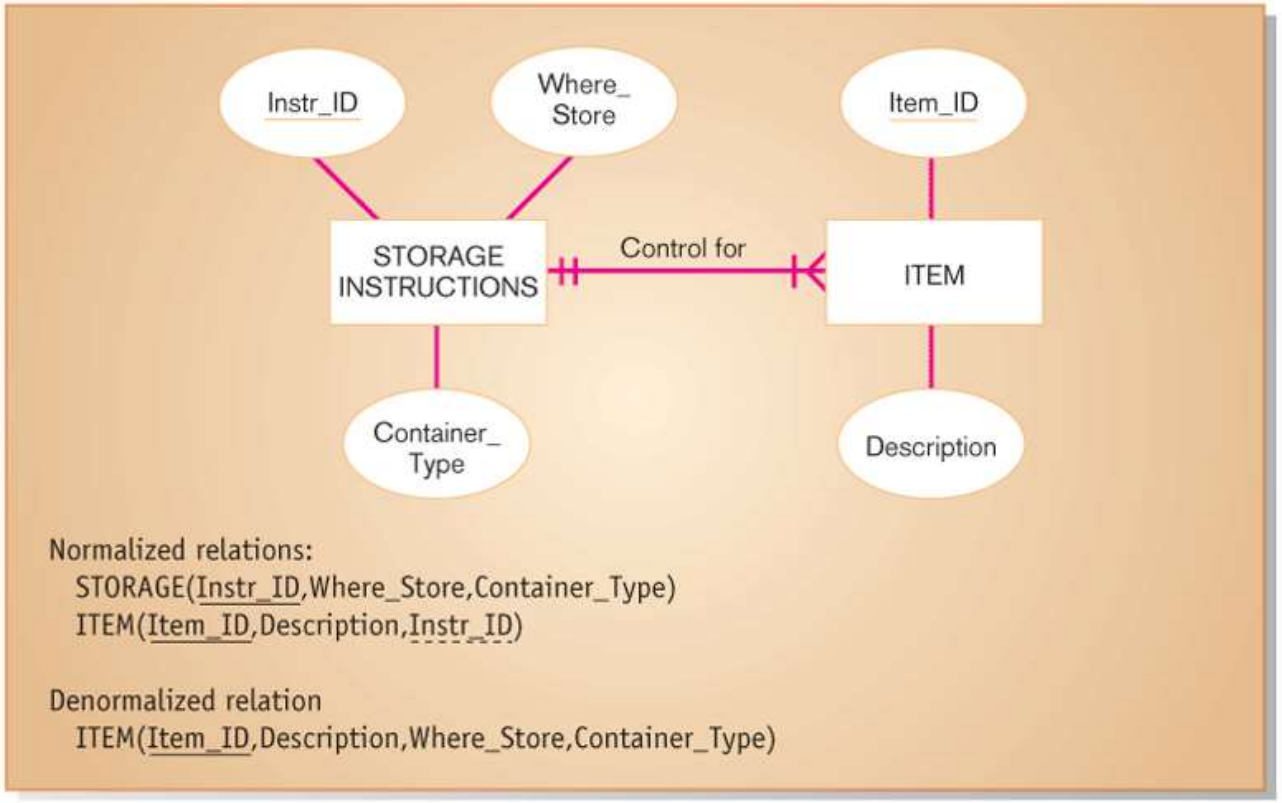
Figure 10-19b Possible denormalization situations - A many-to-many relationship with nonkey attributes



Normalized relations:
VENDOR(Vendor_ID,Address,Contact_Name)
ITEM(Item_ID,Description)
PRICE QUOTE(Vendor_ID,Item_ID,Price)

Denormalized relations:
VENDOR(Vendor_ID,Address,Contact_Name)
ITEM-QUOTE(Vendor_ID,Item_ID,Description,Price)

Figure 10-19c Possible denormalization situations - Reference data



Denormalize with caution

- ✖ Denormalization can
  - ✖ Increase chance of errors and inconsistencies
  - ✖ Reintroduce anomalies
  - ✖ Force reprogramming when business rules change
- ✖ Perhaps other methods could be used to improve performance of joins
  - ✖ Organization of tables in the database (file organization and clustering)
  - ✖ Proper query design and optimization

Designing Physical Tables

- Relational database is a set of related tables
- Physical Table
  - A named set of rows and columns that specifies the fields in each row of the table
- Design Goals
  - Efficient use of secondary storage (disk space)
    - Disks are divided into units that can be read in one machine operation.
    - Space is used most efficiently when the physical length of a table row divides close to evenly with storage unit.
  - Efficient data processing
    - Data are most efficiently processed when stored next to each other in secondary memory.

✖ Physical File:
- + A named portion of secondary memory allocated for the purpose of storing physical records
- + Tablespace–named logical storage unit in which data from multiple tables/views/objects can be stored

+ Tablespace components
- + Segment – a table, index, or partition
- + Extent–contiguous section of disk space
- + Data block – smallest unit of storage

- **File –** A file is named collection of related information that is recorded on secondary storage such as magnetic disks, magnetic tables and optical disks.
- **File Organization**
  - Technique for physically arranging records of a file on secondary storage
  - A database consist of a huge amount of data. The data is grouped within a table in RDBMS, and each table have related records. A user can see that the data is stored in form of tables, but in actual this huge amount of data is stored in physical memory in form of files.
  - File Organization refers to the logical relationships among various records that constitute the file, particularly with respect to the means of identification and access to any specific record. In simple terms, Storing the files in certain order is called file Organization. **File Structure** refers to the format of the label and data blocks and of any logical control record.

Objectives for choosing file organization

1. Fast data retrieval
2. High throughput for processing transactions
3. Efficient use of storage space
4. Protection from failures or data loss
5. Minimizing need for reorganization
6. Accommodating growth
7. Security from unauthorized use

Types of File Organizations

- Sequential File Organizations
- Indexed File Organizations
- Hash File Organizations
- Heap file organization
- B+ file organization
- Indexed sequential access method (ISAM)
- Cluster file organization

Sequential File Organization: In this method the file are stored one after another in a sequential manner.This can be done in two ways:

1.Pile File Organization Methods

- we store the records in a sequence i.e one after other in the order in which they are inserted into the tables.

2.Sorted File Organization Methods

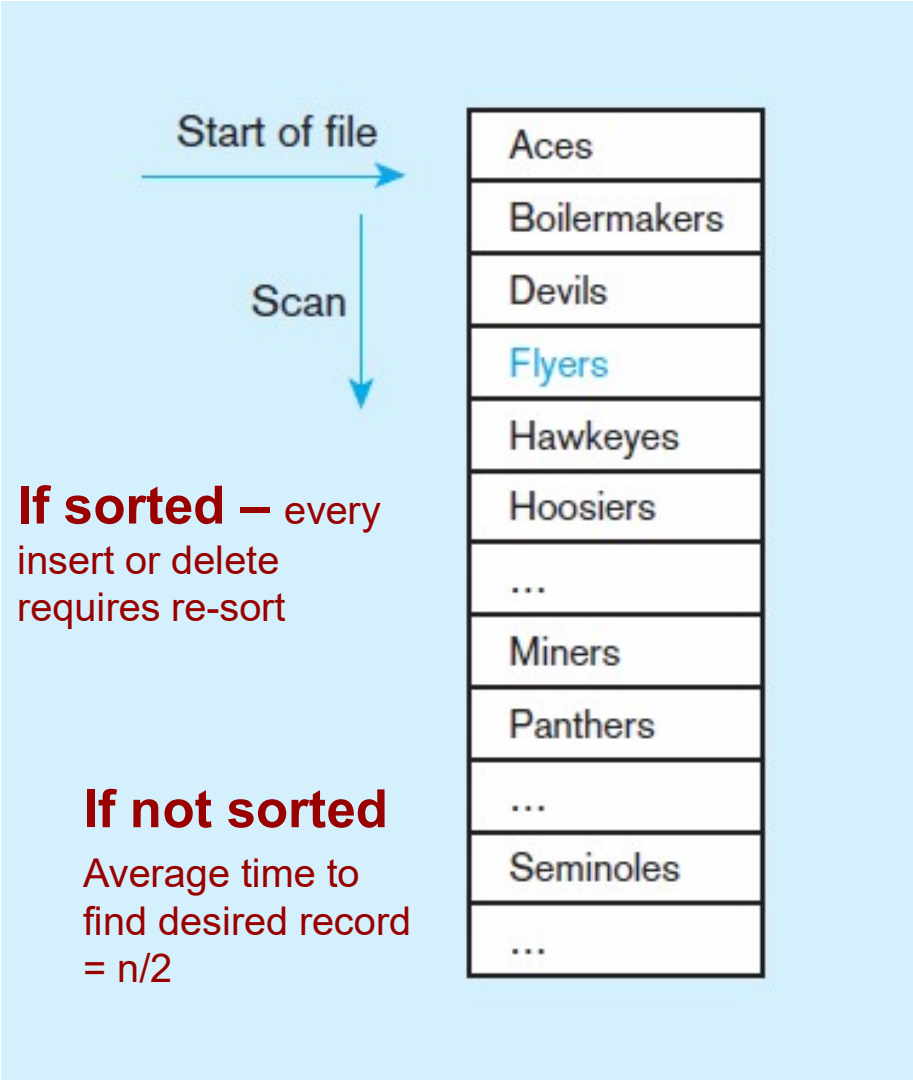
- whenever a new record has to be inserted, it is always inserted in a sorted (ascending or descending) manner.
- Sorting of records may be based on any primary key or any other key.

Pros –

- Fast and efficient method for huge amount of data.
- Simple design.
- Files can be easily stored in magnetic tapes i.e cheaper storage mechanism.

Cons –

- Time wastage as we cannot jump on a particular record that is required, but we have to move in a sequential manner which takes our time.
- Sorted file method is inefficient as it takes time and space for sorting records.



Records of the file are stored in sequence by the primary key field values

Fig:
Sequential file organization

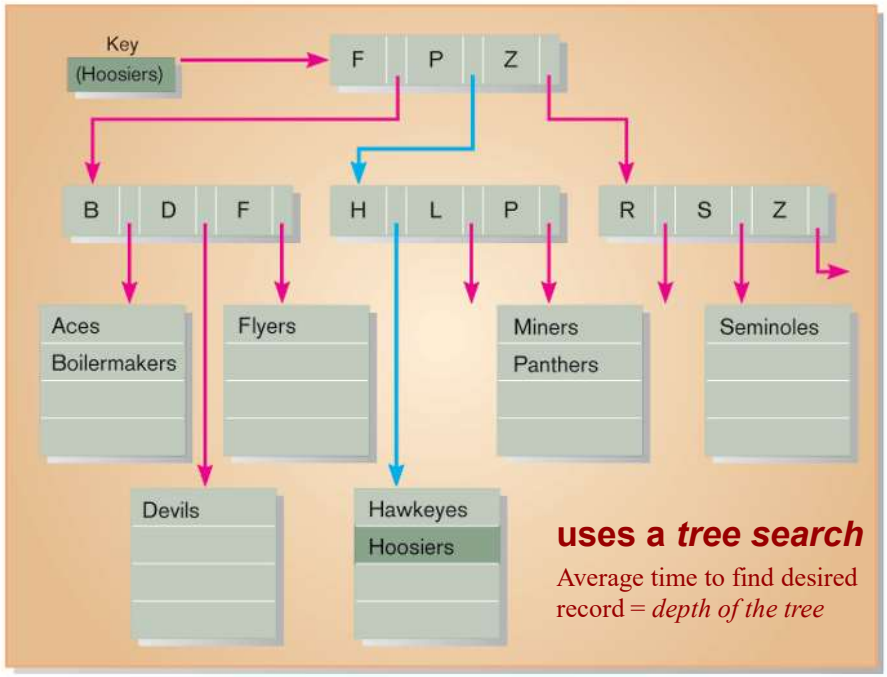
Indexed File Organization

- A file organization in which rows are stored either sequentially or non sequentially and an index is created that allows software to locate individual rows
- Index: A table used to determine the location of rows in a file that satisfy some condition
- Primary keys are automatically indexed
- Other fields or combinations of fields can also be indexed; these are called secondary keys (or nonunique keys)
- **Pros:**
  - Searching and Recording is fast.
- **Cons**
  - requires extra space in the disk to store the index value

Guidelines for Choosing Indexes

- Specify a unique index for the primary key of each table.
- Specify an index for foreign keys.
- Specify an index for nonkey fields that are referenced in
- qualification, sorting and grouping commands for the purpose of retrieving data.

Figure 10-20b Comparison of file organizations - Indexed



Hashed File Organization

- A file organization in which the address for each row is determined using an algorithm.
- Hash File Organization uses the computation of hash function on some fields of the records.
- The hash function's output determines the location of disk block where the records are to be placed.
- there is no effort for searching and sorting the entire file.
- When data access order is random and exact match is to be find that case this techniques works well.
- Sometime hash collision may occurs and need to manage.

Figure 10-20c Comparison of file organizations - Hashed

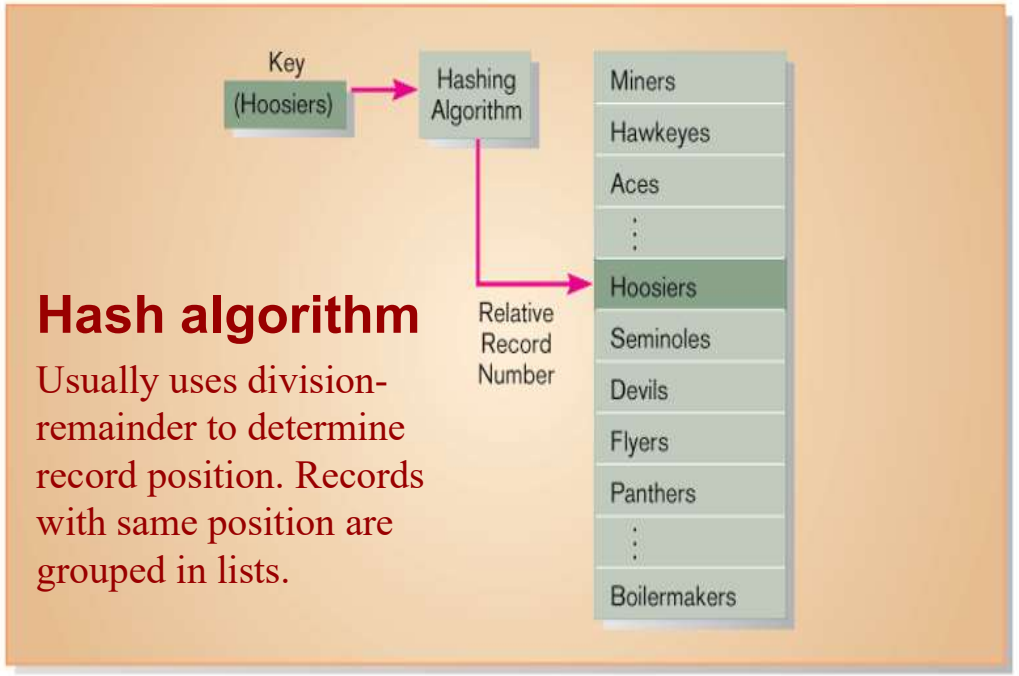


Table 10-3 Comparative Features of Sequential, Indexed, and Hashed File Organizations

	File Organization		
Factor	*Sequential*	*Indexed*	*Hashed*
Storage space	No wasted space	No wasted space for data, but extra space for index	Extra space may be needed to allow for addition and deletion of records
Sequential retrieval on primary key	Very fast	Moderately fast	Impractical
Random retrieval on primary key	Impractical	Moderately fast	Very fast
Multiple key retrieval	Possible, but requires scanning whole file	Very fast with multiple indexes	Not possible
Deleting rows	Can create wasted space or require reorganizing	If space can be dynamically allocated, this is easy, but requires maintenance of indexes	Very easy
Adding rows	Requires rewriting file	If space can be dynamically allocated, this is easy, but requires maintenance of indexes	Very easy, except multiple keys with same address require extra work
Updating rows	Usually requires rewriting file	Easy, but requires maintenance of indexes	Very easy

Using and Selecting Keys

- Creating a unique key index
  - Example: CustomerID (primary key) of Customer

```
CREATE UNIQUE INDEX CustIndex_PK ON Customer_T(CustomerID);
```

  - Example: Composite primary key for OrderLine

```
CREATE UNIQUE INDEX LineIndex_PK ON OrderLine_T(OrderID, ProductID);
```

- Creating a secondary key index
  - Example: Description field for Product (not unique)

```
CREATE INDEX DescIndex_FK ON Product_T(Description);
```

Rules for Using Indexes

1. Use on larger tables
2. Index the primary key of each table
3. Index search fields (fields frequently in WHERE clause)
4. Fields in SQL ORDER BY and GROUP BY commands
5. When there are >100 values but not when there are <30 values
6. Avoid use of indexes for fields with long values; perhaps compress values first
7. If key to index is used to determine location of record, use surrogate (like sequence nbr) to allow even spread in storage area
8. DBMS may have limit on number of indexes per table and number of bytes per indexed field(s)
9. Be careful of indexing attributes with null values; many DBMSs will not recognize null values in an index search

Query Optimization

- ✖ Parallel query processing–possible when working in multiprocessor systems
- ✖ Overriding automatic query optimization–allows for query writers to preempt the automated optimization
- ✖ Data warehouses are already configured for optimized query performance

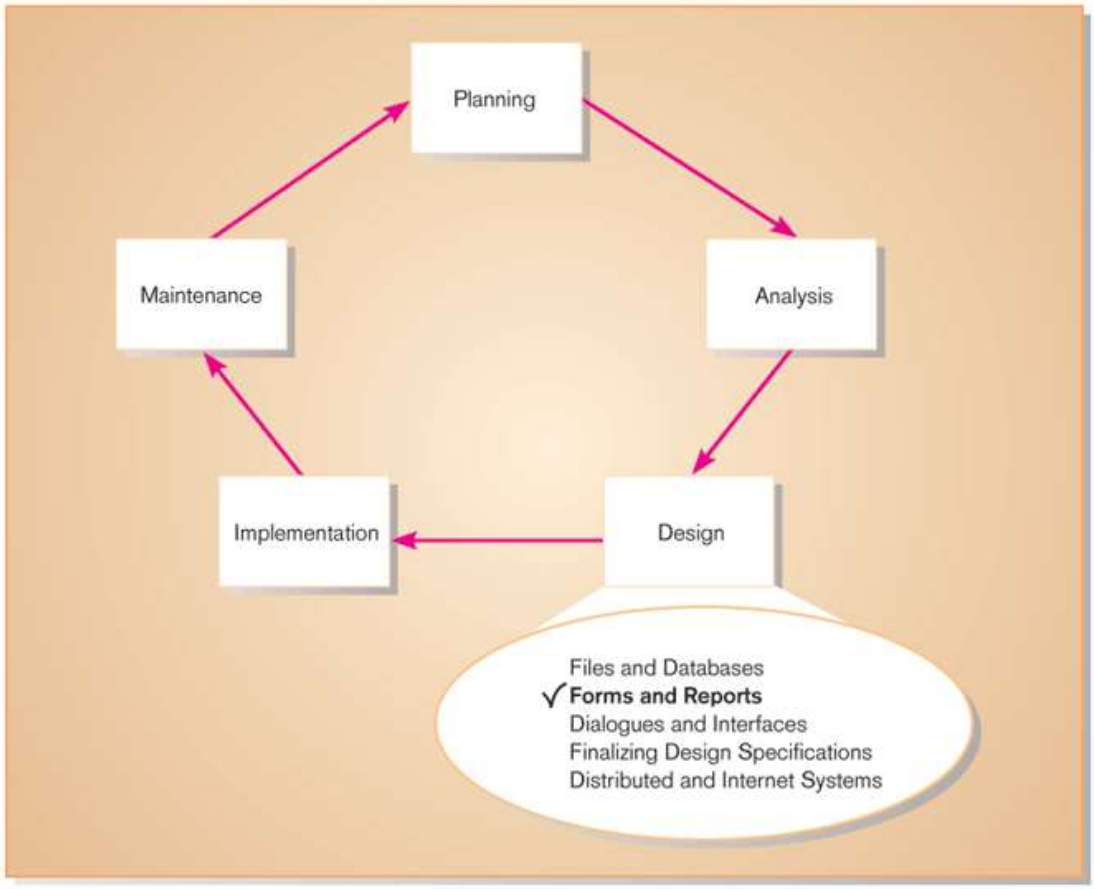
4.2 Designing Forms and Reports

- Introduction;
- Designing Forms and Reports;
- Formatting Forms and Reports;
- Assessing Usability

Learning Objectives

- ✓Explain the process of form and report design.
- ✓Apply general guidelines for formatting forms and reports.
- ✓Use color and know when color improves the usability of information.
- ✓Format text, tables, and lists effectively.
- ✓Explain how to assess usability and describe factors affecting usability.

Figure 11-1 Systems development life cycle with logical design phase highlighted



- **Introduction:**
- **Form**
  - A business document that contains some predefined data and may include some areas where additional data are to be filled in.
  - An instance of a form is typically based on one database record.
  - It also allows us to enter data into the database, display it for review and also print it for distribution.
  - However, an electronic form has several important advantages over standard paper forms.
  - These have the advantage of using a computer database and are more versatile and powerful than paper forms.
  - Examples of forms are Business forms, Electronic spread sheet, ATM transaction layout, etc.
- **Report**
  - A business document that contains only predefined data.
  - A passive document for reading or viewing data.
  - Typically contains data from many database records or transactions.
  - Computer systems use reporting and query applications to retrieve the data that are available in the database and present it in a way that provides useful information, drives decision-making and supports business projects.
  - A report presents data as meaningful information, which can be used and distributed.
  - A report is the information that is organized and formatted to fit the required specification.
  - Reports can be printed on paper, or these may be transferred to a computer file, a visual display screen, etc.
  - Examples of reports are: invoices, weekly sales summaries, mailing labels, pie chart, etc.

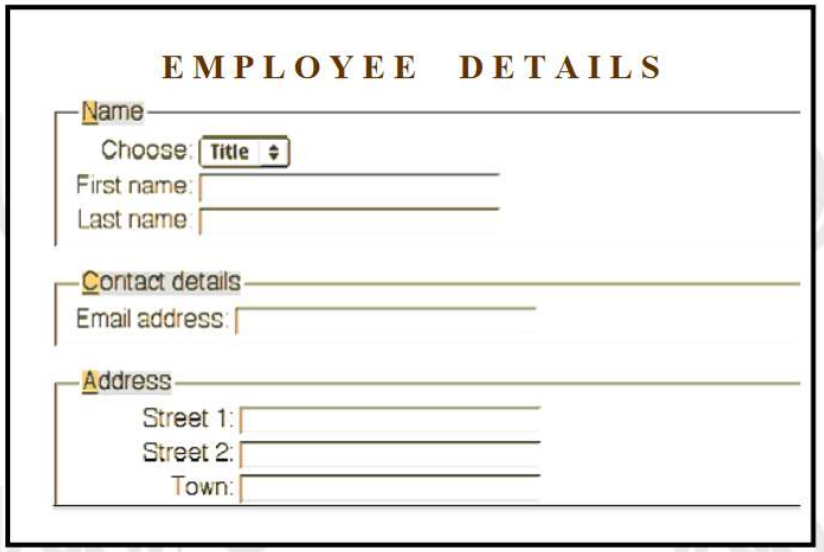


Fig: Simple Employ Form

EMPLOYEE RESIDENCE PHONE LIST

S.No	LAST NAME	FIRST NAME	DESIGNATION	PHONE NUMBER
1	Verma	Ajay	Regional Manager	6522081
2	Gupta	Vinay	Branch Manager	6478017
3	Michael	Nancy	H.R Manager	6152430
4	Singh	Amar	Sales Executive	5769081

Fig: Simple Employ Report

- **Importance of Forms**
  - A form provides an easy way to view data.
  - Using forms, data can be entered easily. This saves time and prevents typographical errors.
  - Forms present data in an attractive format with special fonts and other graphical effects such as colour and shading.
  - Forms offer the most convenient layout for entering, changing and viewing records present in the database.
  - An entry field in a form can present a list of valid values from which users can pick to fill out the field easily.
- **Importance of Reports**
  - We can organize and present data in groups.
  - We can calculate running totals, group totals, grand totals, percentage of totals, etc.
  - Within the body of Reports, we can include sub-forms, sub-reports and graphs.
  - We can present data in an attractive format with pictures, special fonts and lines.
  - We can create a design for a report and save it so that we can use it over and over again.

- **Common Types of Reports:**
  - Scheduled: produced at predefined time intervals for routine information needs
  - Key-indicator: provide summary of critical information on regular basis
  - Exception: highlights data outside of normal operating ranges
  - Drill-down: provide details behind summary of key-indicator or exception reports
  - Ad-hoc: respond to unplanned requests for non-routine information needs

- **Difference between Forms and Reports**
  - Forms can be used for both input and output. Reports, on the other hand, are used for output, i.e., to convey information on a collection of items.
  - Typically, forms contain data from only one record, or are at least based on one record such as data about one student, one customer, etc. A report, on the other hand is only for reading and viewing. So, it often contains data about multiple unrelated records in a computer file or database.
  - Although we can also print forms and datasheets, reports give more control over how data are displayed and show greater flexibility in presenting summary information.

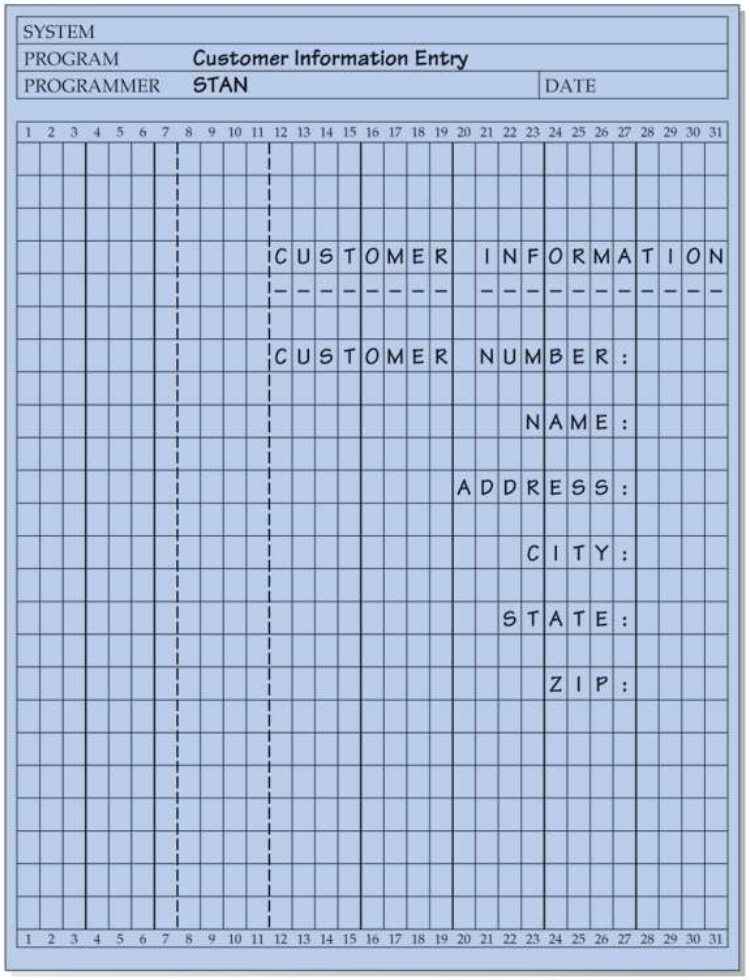
PROCESS OF DESIGNING FORMS AND REPORTS

- Good quality business processes deliver the right information to the right people in the right format and at the right time.
- The design of forms and reports concentrates on this goal.
- Designing of forms and reports is a user-focused activity that typically follows a prototyping approach.
- **Step 1:Requirement Determination:**
  - we should have a clear idea so as to what is the aim of the form or report and what information is to be collected from the user.
  - There are some useful questions related to the creation of all forms and reports, such as “who, what, when, where and how” which must be answered in order to design effective forms and reports.
  - **WHO ->** Understanding who the actual users are, their skills and abilities, their education level, business background, etc., will greatly enhance the ability to create effective design.
  - **WHAT ->** We need to have a clear understanding of what is the purpose of the form or report and what task will the users be performing and what information is required so as to successfully complete the given task.
  - **WHEN ->** Knowing when exactly the form or report is needed and used will help to set up time limits so that the form or report can be made available to the users within that time frame.
  - **WHERE ->**Where will the users be using this form or report (i.e., will the users have access to on-line systems or will they be using them in the field)?
  - **HOW ->**How many people will be using this form or report, i.e., if the form or report is to be used by a single person, then it will be simple in design but if a large number of people are going to use it, then the design will have to go through a more extensive requirements collection and usability assessment process.

PROCESS OF DESIGNING FORMS AND REPORTS

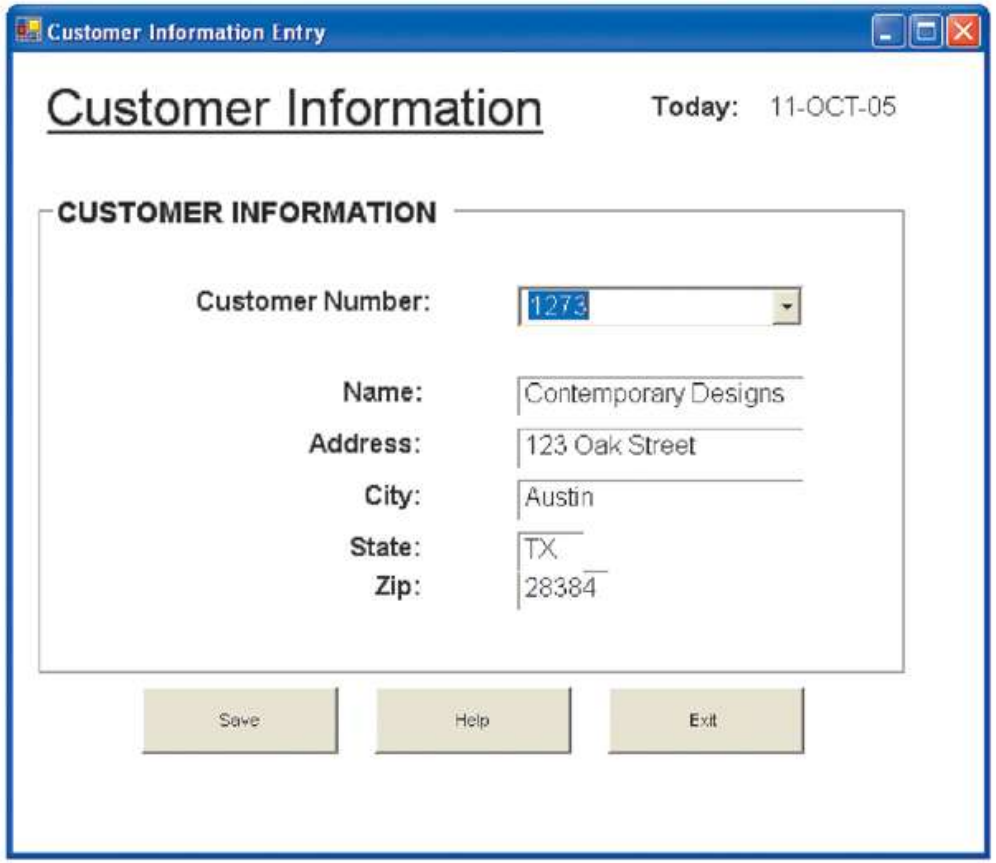
- Step 2:Prototyping
  - Initial prototype is designed from requirements
  - Users review prototype design and either accept the design or request changes
  - If changes are requested, the construction-evaluation-refinement cycle is repeated until the design is accepted
- Step 3:Design, Validate and Test the outputs using some combination of the following tools:
  - a)Layout tools (Ex.: Hand sketches, printer/display layout charts or CASE)
    b)Prototyping tools (Ex.: Spreadsheet, PC, DBMS, 4GL)
  - c)Code generating tools (Ex.: Report writer)

Figure 11-2 The layout of a data input form using a coding sheet



A coding sheet is an "old" tool for designing forms and reports, usually associated with text-based forms and reports for mainframe applications.

Figure 11-3
A data input screen designed in Microsoft's Visual Basic .NET



Visual Basic and other development tools provide computer aided GUI form and report generation.

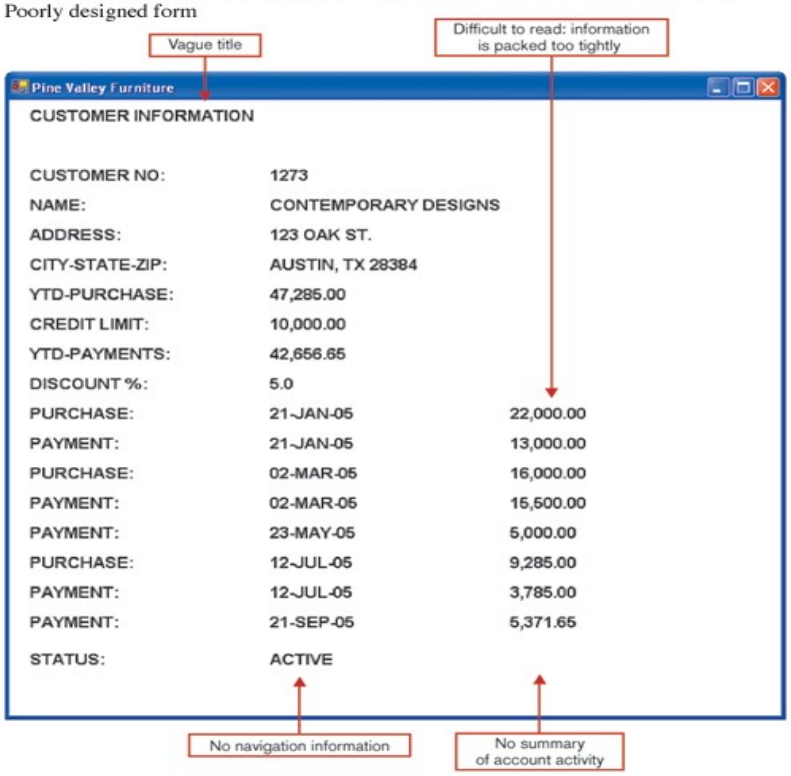
Form/Report Design Specification

- The major deliverable of interface design
- Involves three parts:
  - **Narrative overview: characterizes users, tasks, system, and environmental factors.**
    - This contains the general overview of the characteristics of actual users of the form or report, task, the system that will be used and the environment factors in which the form or report will be used.
    - The main purpose of Narrative overview is to provide information in detail to the people who will develop the final form or report, regarding the main aim of the form, who the actual users of the form will be, and how it will be used, so that they can make appropriate implementation decisions.
  - **Sample design: image of the form (from coding sheet or form building development tool)**
    - The sample design of the form may be hand-drawn using a coding sheet or it may be developed using CASE or standard development tools.
    - If the sample design is done using actual development tools, then the form can be thoroughly tested and assessed.
  - **Assessment: measuring test/usability results (consistency, sufficiency, accuracy, etc.)**
    - This section provides information required for testing and usability assessment. While testing, it is important to use realistic or reasonable data and demonstrate all controls.

Guidelines for Form and Report Design

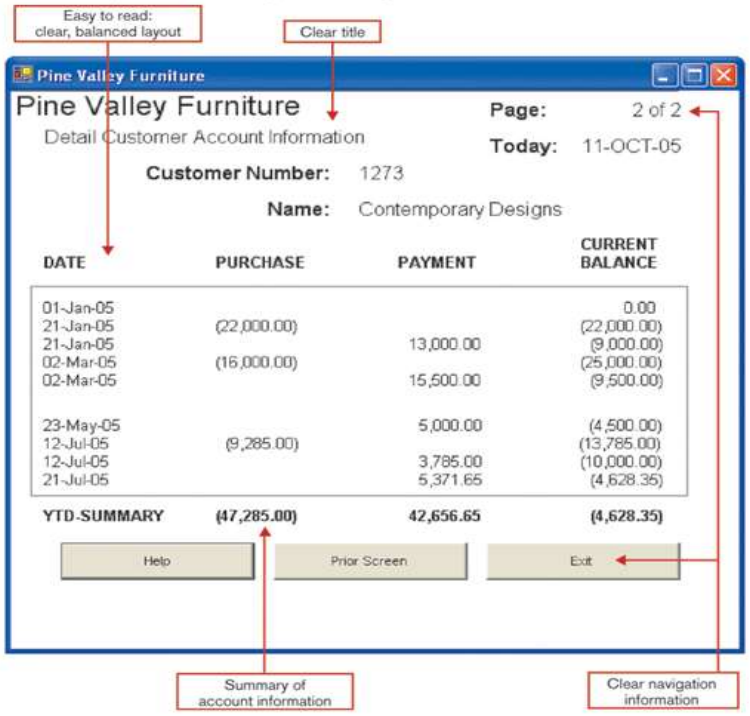
- Meaningful titles: clear, specific, version information, current date
- Meaningful information– include only necessary information, with no need to modify
- Balanced layout: adequate spacing, margins, and clear labels
- Easy navigation system: show how to move forward and backward, and where you are currently

Figure 11-5a Contrasting customer information forms (Pine Valley Furniture) - Poorly designed form



A poor form design

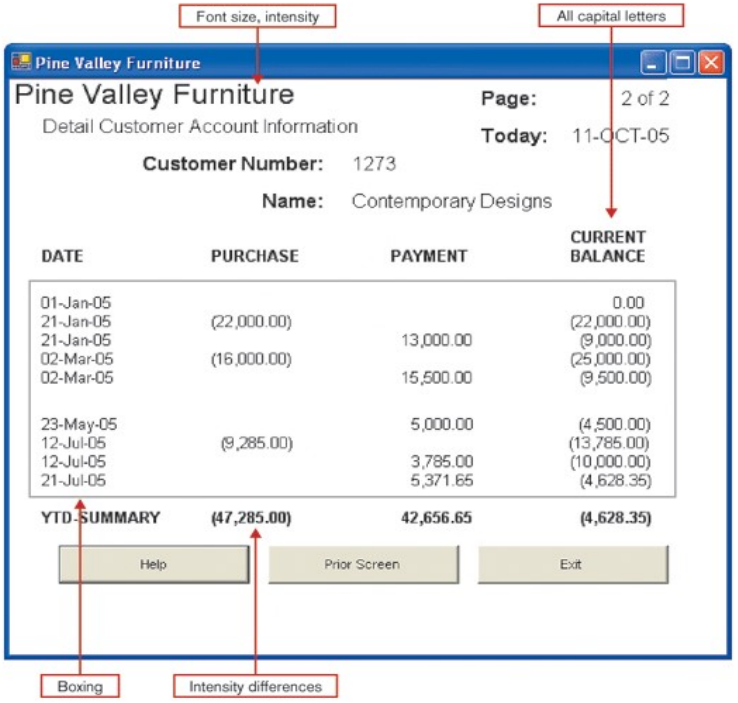
Figure 11-5b Contrasting customer information forms (Pine Valley Furniture) - Improved design for form



A better form design

- **Uses of Highlighting in Forms and Reports**
  - Notify users of errors in data entry or processing.
  - Provide warnings regarding possible problems.
  - Draw attention to keywords, commands, high-priority messages, unusual data values.
- **Methods for Highlighting**
  - Blinking
  - Audible tones
  - Intensity differences
  - Size differences
  - Font differences
  - Reverse video
  - Boxing
  - Underlining
  - All capital letters
  - Offset positions of nonstandard information

Figure 11-6 Customer account status display using various highlighting techniques (Pine Valley Furniture)



Highlighting can include use of upper case, font size differences, bold, italics, underline, boxing, and other approaches.

- **Color Vs No Color**

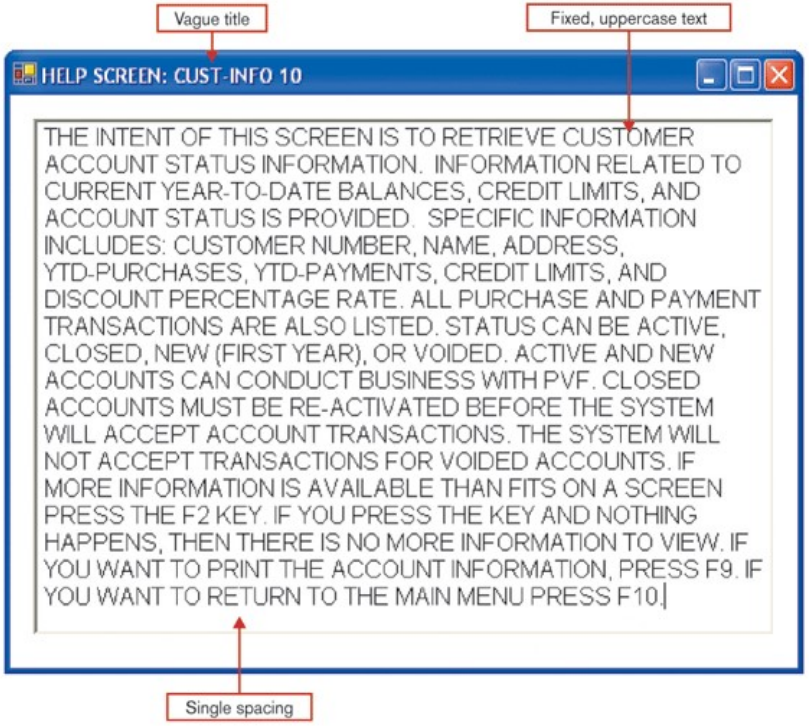
- Benefits from Using Color
  - Soothes or strikes the eye
  - Accents an uninteresting display
  - Facilitates subtle discriminations in complex displays
  - Emphasizes the logical organization of information
  - Draws attention to warnings
  - Evokes more emotional reactions

- Problems from Using Color
  - Color pairings may wash out or cause problems for some users
  - Resolution may degrade with different displays
  - Color fidelity may degrade on different displays
  - Printing or conversion to other media may not easily translate

Guidelines for Displaying Text

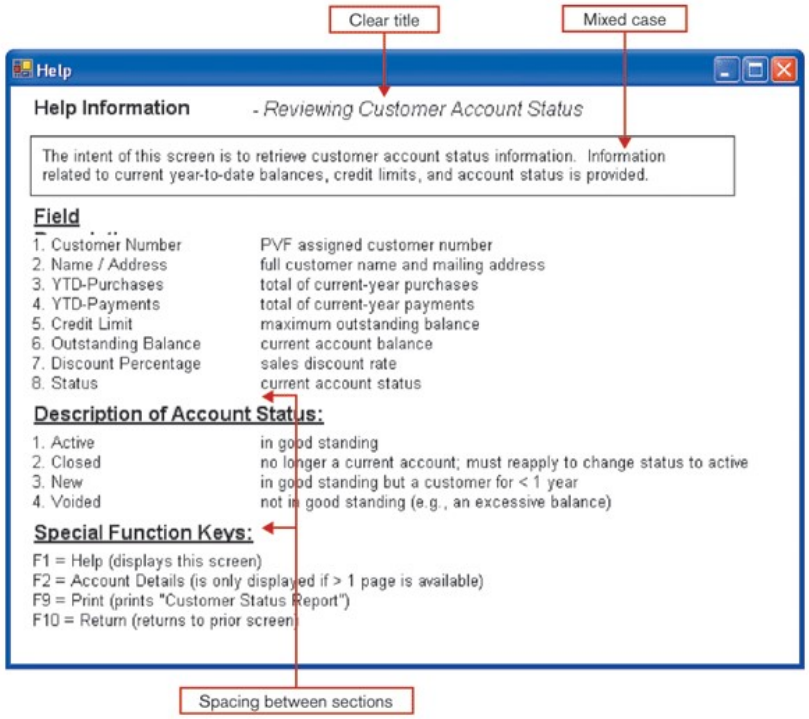
- Case: mixed upper and lower case, use conventional punctuation
- Spacing: double spacing if possible, otherwise blank lines between paragraphs
- Justification: left justify text, ragged right margins
- Hyphenation: no hyphenated words between lines
- Abbreviations: only when widely understood and significantly shorter than full text

Figure 11-7a Contrasting the display of textual help information - Poorly designed help screen with many violations of the general guidelines for displaying text



A poor help screen design

Figure 11-7b Contrasting the display of textual help information - An improved design for a help screen

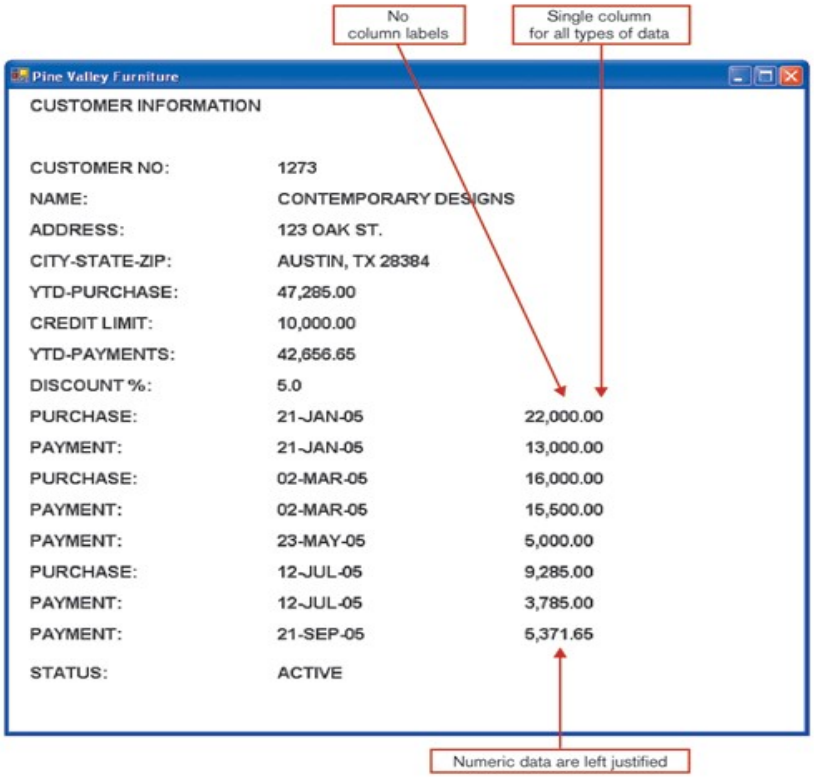


A better help screen design

Guidelines for Tables and Lists

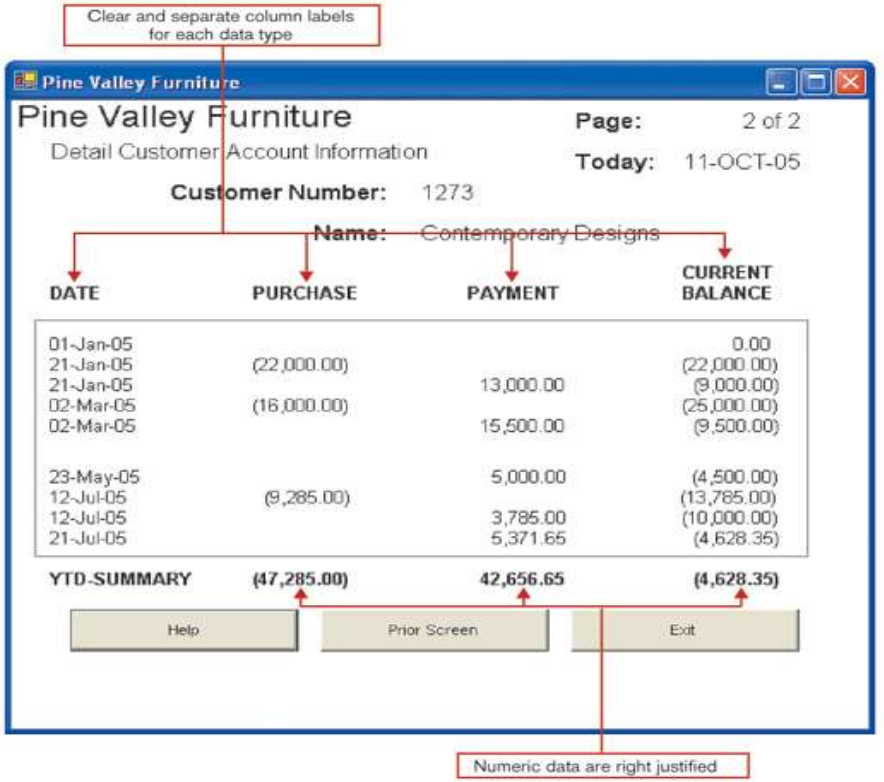
- Labels
  - All columns and rows should have meaningful labels.
  - Labels should be separated from other information by using highlighting.
  - Redisplay labels when the data extend beyond a single screen or page.
- Formatting columns, rows and text:
  - Sort in a meaningful order.
  - Place a blank line between every five rows in long columns.
  - Similar information displayed in multiple columns should be sorted vertically.
  - Columns should have at least two spaces between them.
  - Allow white space on printed reports for user to write notes.
  - Use a single typeface, except for emphasis.
  - Use same family of typefaces within and across displays and reports.
  - Avoid overly fancy fonts.
- Formatting numeric, textual and alphanumeric data:
  - Right justify numeric data and align columns by decimal points or other delimiter.
  - Left justify textual data. Use short line length, usually 30 to 40 characters per line.
  - Break long sequences of alphanumeric data into small groups of three to four characters each.

Figure 11-8a Contrasting the display of tables and lists (Pine Valley Furniture) - Poorly designed form



A poor table design

Figure 11-8b Contrasting the display of tables and lists (Pine Valley Furniture) - Improved design for form



A better table design

- **Table vs Graphs**
  - Use tables for reading individual data values
  - Use graphs for:
    - Providing quick summary
    - Displaying trends over time
    - Comparing points and patterns of variables
    - Forecasting activity
    - Simple reporting of vast quantities of information

Figure 11-9 Tabular report illustrating numerous design guidelines (Pine Valley Furniture)

Place meaningful labels on all columns and rows

Alphabetic text is left justified

Use a meaningful title

Box the table data to improve the appearance of the table

Pine Valley Furniture
Salesperson Annual Summary Report, 2004

January 10, 2005 Page 1 of 2

Region	Salesperson	SSN	Quarterly Actual Sales First	Second	Third	Fourth
Northwest & Mountain						
	Baker	999-99-9999	195,000	146,000	133,000	120,000
	Hawthorne	999-99-9999	220,000	175,000	213,000	198,000
	Hodges	999-99-9999	110,000	95,000	170,000	120,000
Midwest & Mid-Atlantic						
	Franklin	999-99-9999	110,000	120,000	170,000	90,000
	Stephenson[1]	999-99-9999	75,000	66,000	80,000	80,000
	Swenson	999-99-9999	110,000	98,000	100,000	90,000
New England						
	Brightman	999-99-9999	250,000	280,000	260,000	330,000
	Kennedy	999-99-9999	310,000	190,000	270,000	280,000

[1]Sales reflect July 1, 2004 – December 31, 2004.

Superscript characters can be used to alert reader of more detailed information

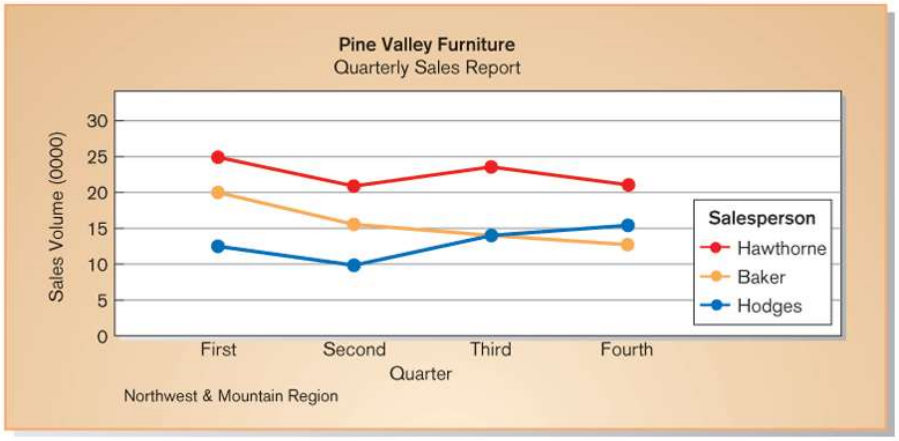
Sort columns in some meaningful order (names are sorted alphabetically within region)

Long sequence of alphanumeric data is grouped into smaller segments

Right justify all numeric data

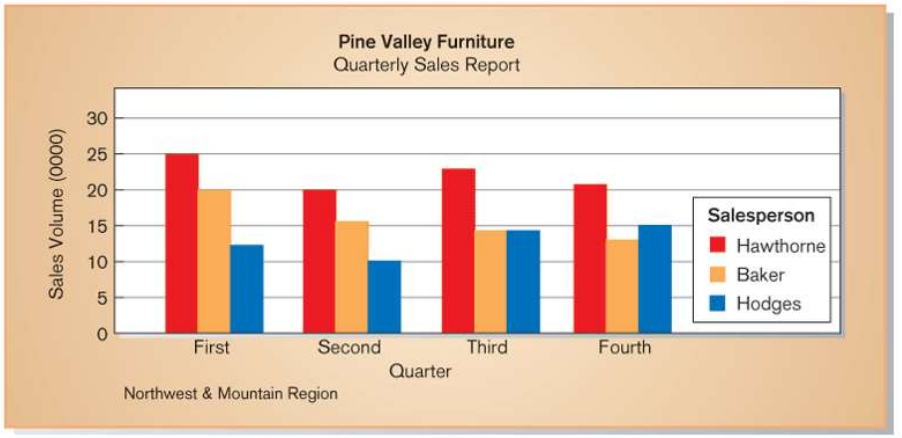
Try to fit table onto a single page to help in making comparisons

Figure 11-10a Graphs for comparison - Line graph



Bar and line graphs give pictorial summary information that can enhance reports and forms.

Figure 11-10b Graphs for comparison - Bar graph



Assessing Usability

- Overall evaluation of how a system performs in supporting a particular user for a particular task
- There are three characteristics
    1. Speed
    2. Accuracy
    3. Satisfaction
- **Guidelines for Maximizing Usability**
    - Consistency: of terminology, formatting, titles, navigation, response time
    - Efficiency: minimize required user actions
    - Ease: self-explanatory outputs and labels
    - Format: appropriate display of data and symbols
    - Flexibility: maximize user options for data input according to preference
- **Characteristics for Consideration**
    - User: experience, skills, motivation, education, personality
    - Task: time pressure, cost of errors, work durations
    - System: platform
    - Environment: social and physical issues

Methods for Assessing Usability

- Time to learn
- Speed of performance
- Rate of errors
- Retention over time
- Subjective satisfaction

Errors in Web Page Layout Design

- Non-standard widgets
- Appearance of advertising
- Bleeding edge technology [**Bleeding edge technology** is a category of technologies so new that they could have a high risk of being unreliable and lead adopters to incur greater expense in order to make use of them. Eg;email]
- Scrolling text and looping animations
- Outdated information
- Slow download times
- Fixed formatted text
- Long pages

Good Web Design Practices

- Lightweight Graphics: small images to quick image download
- Forms and Data Integrity
- Template-based HTML
  - Templates to display and process common attributes of higher-level, more abstract items
  - Creates an interface that is very easy to maintain

Designing Interfaces and Dialogues

Designing Interfaces and Dialogues

- User-focused activity
- Prototyping methodology of iteratively:
  - Collecting information
  - Constructing a prototype
  - Assessing usability
  - Making refinements
- Must answer the who, what, where, and how questions

Designing Interfaces and Dialogues (Cont.)

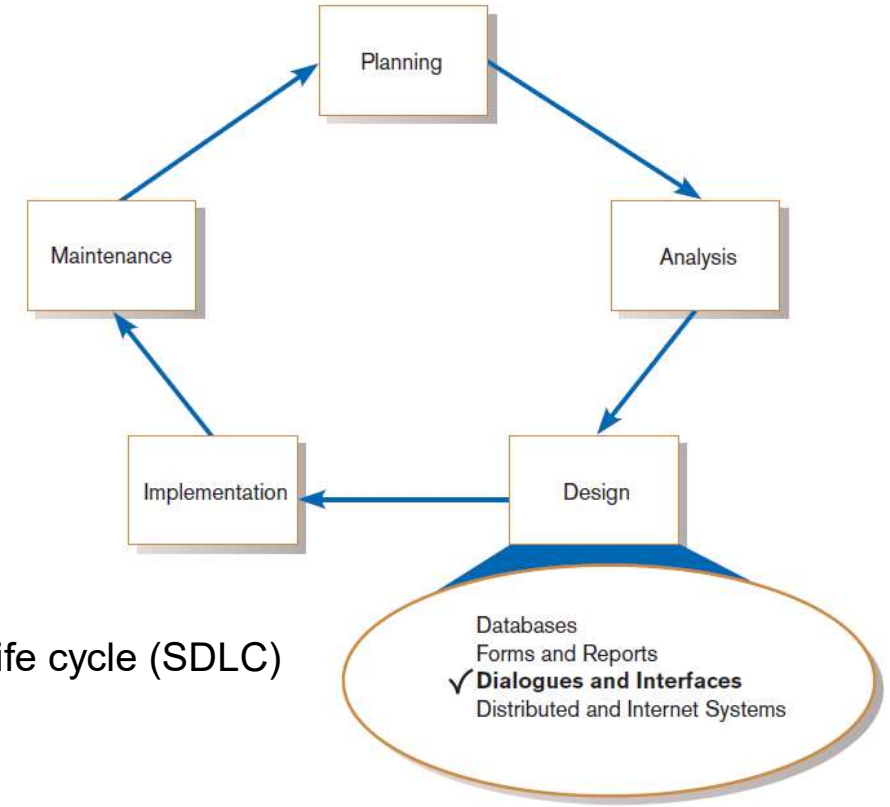


FIGURE 11-1
Systems development life cycle (SDLC)

Deliverables and Outcomes

- Creation of a design specification
  - A typical interface/dialogue design specification is similar to form design, but includes multiple forms and dialogue sequence specifications.
- The specification includes:
  - Narrative overview
  - Sample design
  - Testing and usability assessment
  - Dialogue sequence
- *Dialogue sequence*—the ways a user can move from one display to another

Design Specification

1. Narrative Overview
   a. Interface/Dialogue Name
   b. User Characteristics
   c. Task Characteristics
   d. System Characteristics
   e. Environmental Characteristics

2. Interface/Dialogue Designs
   a. Form/Report Designs
   b. Dialogue Sequence Diagram(s) and Narrative Description

3. Testing and Usability Assessment
   a. Testing Objectives
   b. Testing Procedures
   c. Testing Results
      i) Time to Learn
      ii) Speed of Performance
      iii) Rate of Errors
      iv) Retention over Time
      v) User Satisfaction and Other Perceptions

Figure 11-2
Specification outline for the design of interfaces and dialogues

Interaction Methods and Devices

- **Interface**: a method by which users interact with an information system
- All human-computer interfaces must:
  - have an interaction style, and
  - use some hardware device(s) for supporting this interaction.
- Methods of Interacting
  - Command line
    - Includes keyboard shortcuts and function keys
  - Menu
  - Form
  - Object-based
  - Natural language

Command Language Interaction

- **Command language interaction:** a human-computer interaction method whereby users enter explicit statements into a system to invoke operations
- Example from MS DOS:
  - COPY C:PAPER.DOC A:PAPER.DOC
  - Command copies a file from C: drive to A: drive
- **Menu Interaction:**
  - **Menu interaction**: a human-computer interaction method in which a list of system options is provided and a specific command is invoked by user selection of a menu option
  - **Pop-up menu**: a menu-positioning method that places a menu near the current cursor position
  - **Drop-down menu** is a menu-positioning method that places the access point of the menu near the top line of the display.
    - When accessed, menus open by dropping down onto the display.
    - Visual editing tools help designers construct menus.

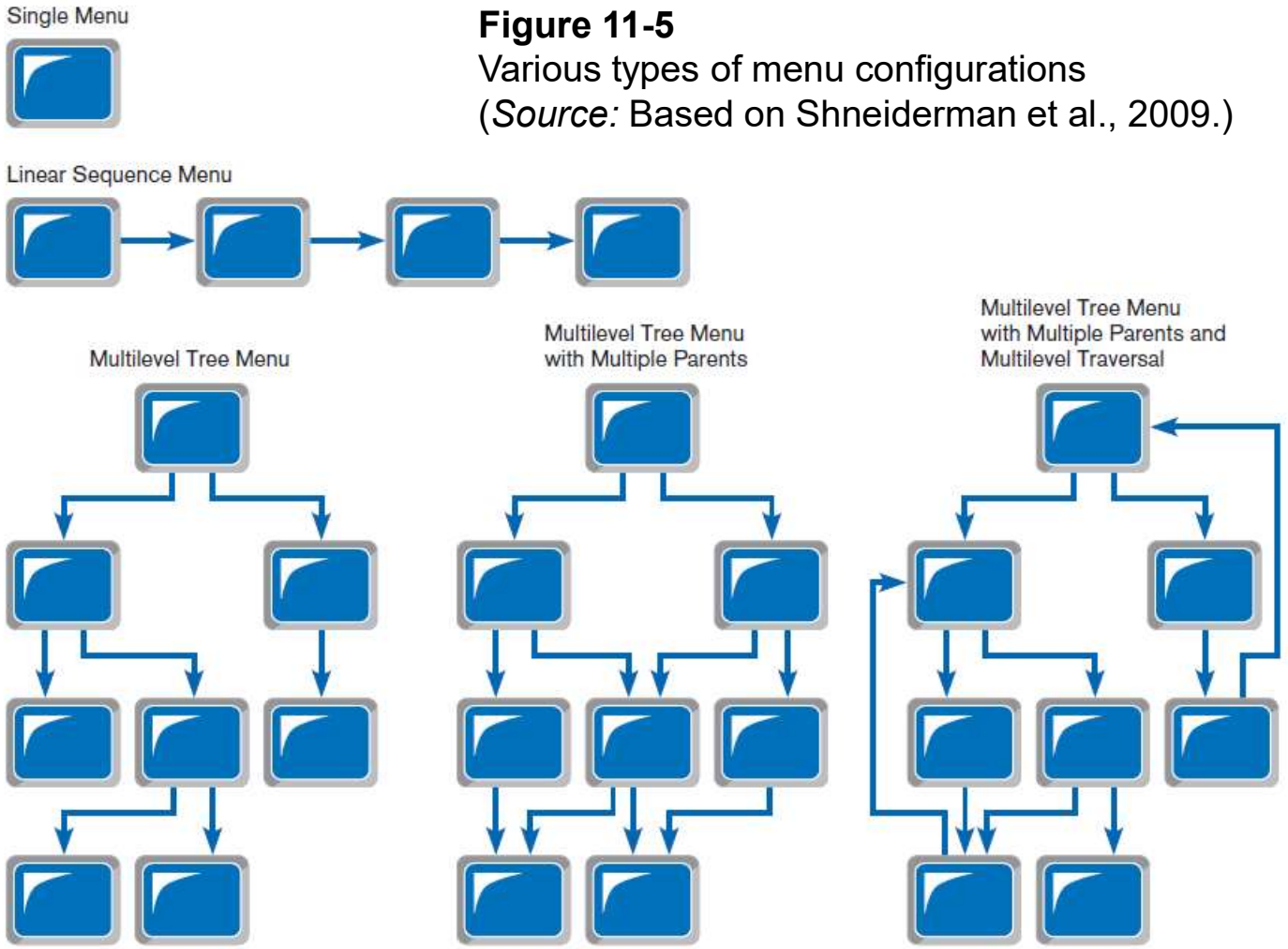


Figure 11-5
Various types of menu configurations
(*Source:* Based on Shneiderman et al., 2009.)

Menu Interaction (Cont.)

- Guidelines for Menu Design
  - **Wording** — meaningful titles, clear command verbs, mixed upper/lower case
  - **Organization** — consistent organizing principle
  - **Length** — all choices fit within screen length
  - **Selection** — consistent, clear and easy selection methods
  - **Highlighting** — only for selected options or unavailable options

Menu Interaction (Cont.)

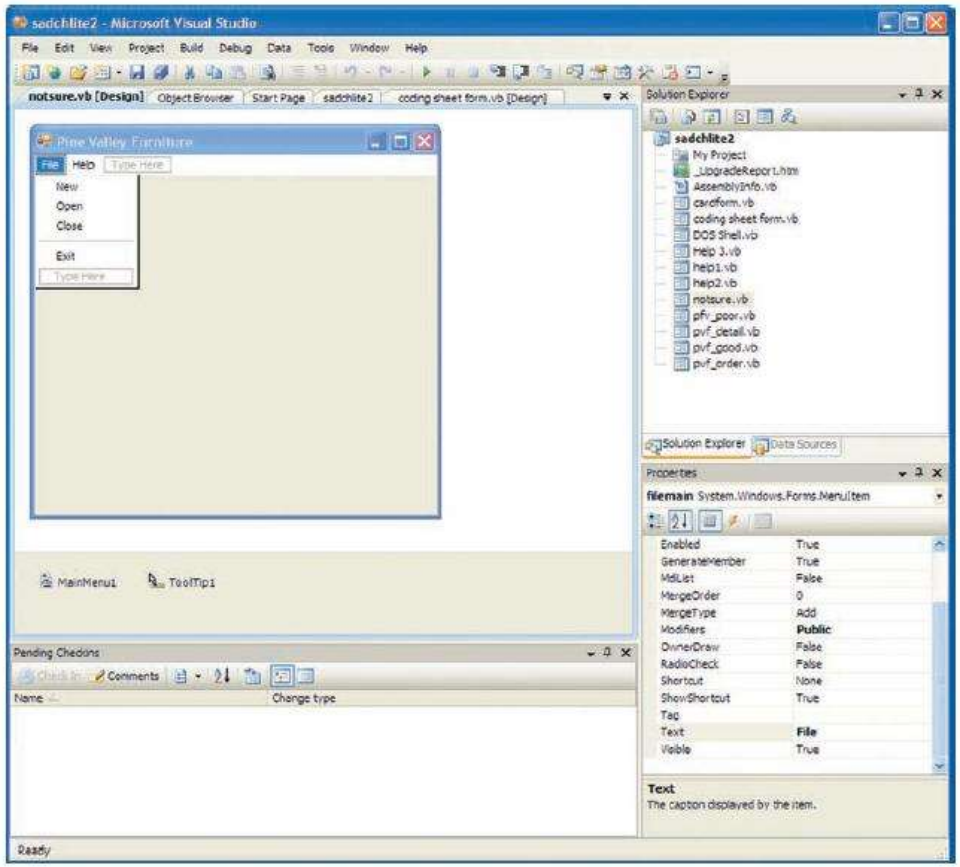


FIGURE 11-8
Menu building with Microsoft Visual Basic .NET

Form Interaction

- **Form interaction**: a highly intuitive human-computer interaction method whereby data fields are formatted in a manner similar to paper-based forms
  - Allows users to fill in the blanks when working with a system.

Form Interaction (Cont.)

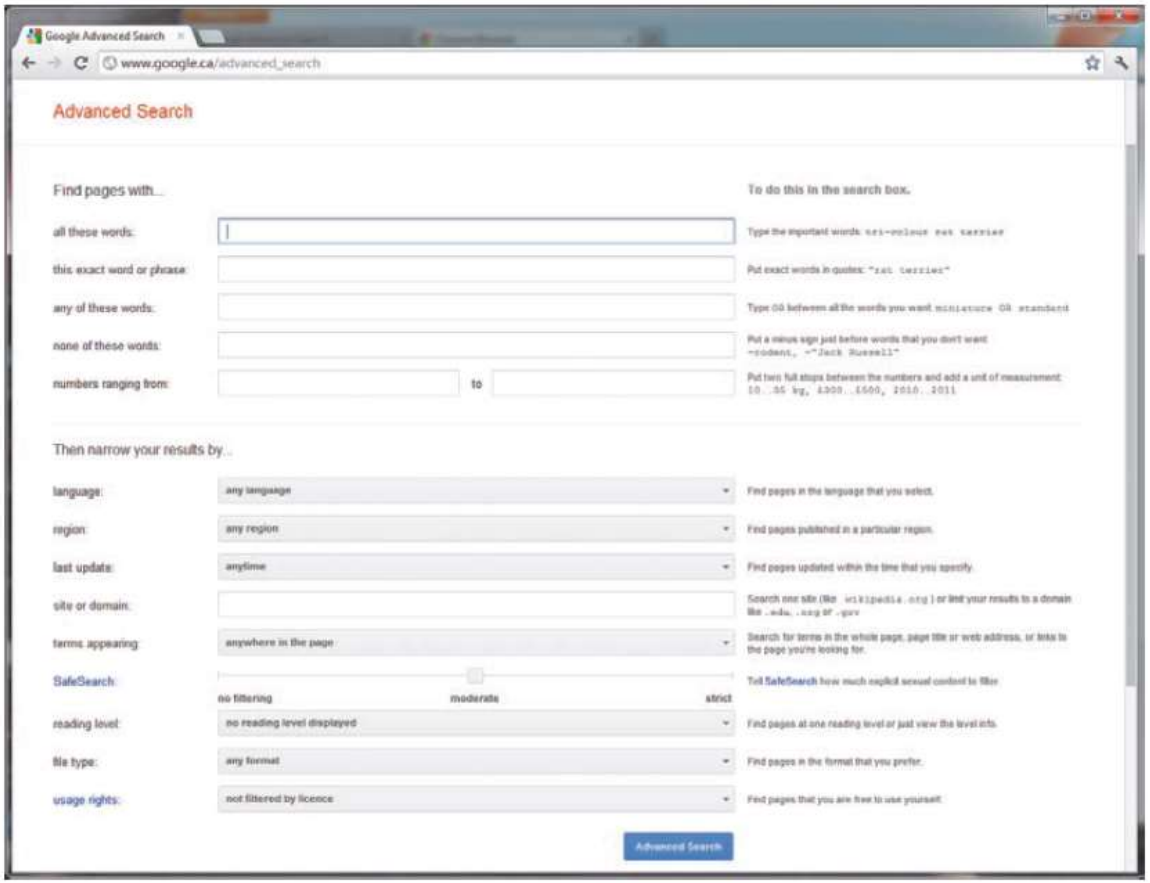


FIGURE 11-9
Example of form interaction from the Google Advanced Search Engine (*Source: Google.)*

Object-Based Interaction

- **Object-based interaction**: a human-computer interaction method in which symbols are used to represent commands or functions
- **Icons**: graphical pictures that represent specific functions within a system
  - Use little screen space and are easily understood by users

Object-Based Interaction (Cont.)

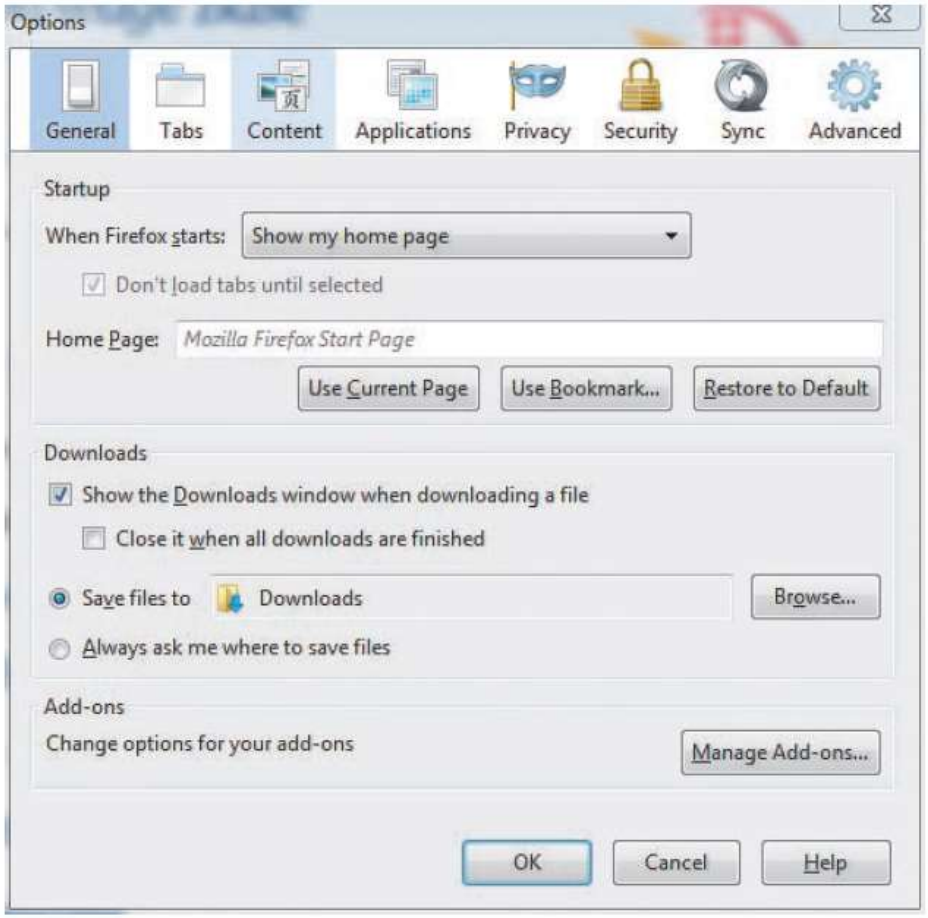


Figure 11-10
Object-based (icon) interface from the Option menu in the Firefox Web browser

Natural Language Interaction

- **Natural language interaction**: a human-computer interaction method whereby inputs to and outputs from a computer-based application are in a conventional spoken language such as English
- Based on research in artificial intelligence
- Current implementations are tedious and difficult to work with, not as viable as other interaction methods.

Hardware Options for System Interaction

- Keyboard
- Mouse
- Joystick
- Trackball
- Touch screen
- Light Pen
- Graphics Tablet
- Voice

Usability Problems with Hardware Devices

- Visual Blocking
  - Extent to which device blocks display when using
- User Fatigue
  - Potential for fatigue over long use
- Movement Scaling
  - Extent to which device movement translates to equivalent screen movement
- Durability
  - Lack of durability or need for maintenance (e.g., cleaning) over extended use

Usability Problems with Hardware Devices (Cont.)

- Adequate Feedback
  - Extent to which device provides adequate feedback for each operation
- Speed
  - Cursor movement speed
- Pointing Accuracy
  - Ability to precisely direct cursor

(*Source:* Based on Blattner and Schultz, 1988.)

Usability Problems with Hardware Devices (Cont.)

TABLE 11-3 Summary of Interaction Device Usability Problems

Device	Problem						
	Visual Blocking	User Fatigue	Movement Scaling	Durability	Adequate Feedback	Speed	Pointing Accuracy
Keyboard	□	□	■	□	■	■	□
Mouse	□	□	■	□	■	□	□
Joystick	□	□	■	□	■	□	■
Trackball	□	□	■	■	■	□	□
Touch Screen	■	■	□	■	□	□	■
Light Pen	■	■	□	□	□	□	■
Graphics Tablet	□	□	■	□	■	□	□
Voice	□	□	■	□	■	□	■

Key:

□ = little or no usability problems

■ = potentially high usability problems for some applications

Usability Problems with Hardware Devices (Cont.)

TABLE 11-4 **Summary of General Conclusions from Experimental Comparisons of Input Devices in Relation to Specific Task Activities**

Task	Most Accurate	Shortest Positioning	Most Preferred
Target Selection	trackball, graphics tablet, mouse, joystick	touch screen, light pen, mouse, graphics tablet, trackball	touch screen, light pen
Text Selection	mouse	mouse	—
Data Entry	light pen	light pen	—
Cursor Positioning	—	light pen	—
Text Correction	light pen, cursor keys	light pen	light pen
Menu Selection	touch screen	—	keyboard, touch screen

Key:

Target Selection = moving the cursor to select a figure or item
Text Selection = moving the cursor to select a block of text
Data Entry = entering information of any type into a system
Cursor Positioning = moving the cursor to a specific position
Text Correction = moving the cursor to a location to make a text correction
Menu Selection = activating a menu item
— = no clear conclusion from the research

(*Source*: Based on Blattner and Schultz, 1988.)

Designing Interfaces

- Forms have several general areas in common:
  - Header information
  - Sequence and time-related information
  - Instruction or formatting information
  - Body or data details
  - Totals or data summary
  - Authorization or signatures
  - Comments

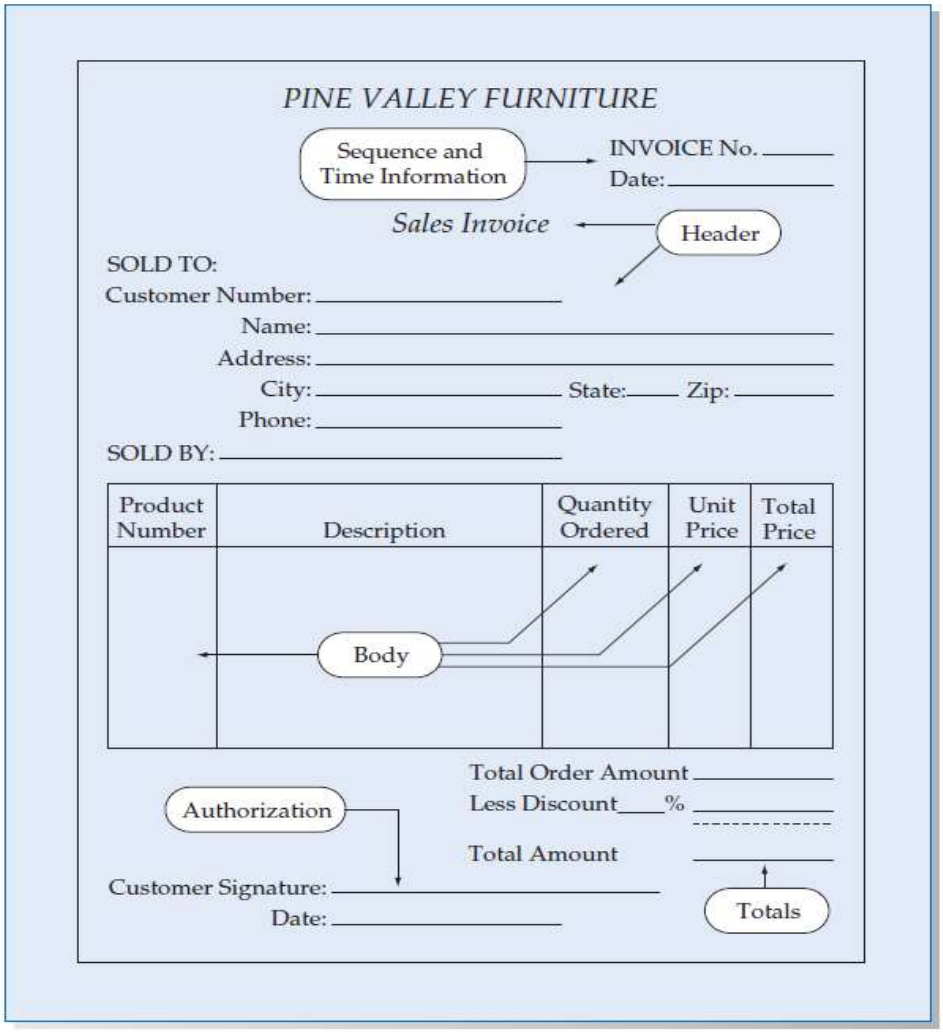


Figure 11-11
Paper-based form for reporting customer sales activity (Pine Valley Furniture)

Designing Interfaces (Cont.)

- Use standard formats similar to paper-based forms and reports.
- Use left-to-right, top-to-bottom navigation.
- Flexibility and consistency:
  - Free movement between fields
  - No permanent data storage until the user requests
  - Each key and command assigned to one function

Structuring Data Entry

Entry	Never require data that are already online or that can be computed
Defaults	Always provide default values when appropriate
Units	Make clear the type of data units requested for entry
Replacement	Use character replacement when appropriate
Captioning	Always place a caption adjacent to fields
Format	Provide formatting examples
Justify	Automatically justify data entries
Help	Provide context-sensitive help when appropriate

Controlling Data Input

- Objective: Reduce data entry errors
- Common sources of data entry errors in a field:
  - Appending: adding additional characters
  - Truncating: losing characters
  - Transcripting: entering invalid data
  - Transposing: reversing sequence of characters

TABLE 11-9 Validation Tests and Techniques to Enhance the Validity of Data Input

Validation Test	Description
Class or Composition	Test to ensure that data are of proper type (e.g., all numeric, all alphabetic, all alphanumeric)
Combinations	Test to see if the value combinations of two or more data fields are appropriate or make sense (e.g., does the quantity sold make sense given the type of product?)
Expected Values	Test to see if data are what is expected (e.g., match with existing customer names, payment amount, etc.)
Missing Data	Test for existence of data items in all fields of a record (e.g., is there a quantity field on each line item of a customer order?)
Pictures/Templates	Test to ensure that data conform to a standard format (e.g., are hyphens in the right places for a student ID number?)
Range	Test to ensure data are within proper range of values (e.g., is a student's grade point average between 0 and 4.0?)
Reasonableness	Test to ensure data are reasonable for situation (e.g., pay rate for a specific type of employee)
Self-Checking Digits	Test where an extra digit is added to a numeric field in which its value is derived using a standard formula (see Figure 11-14)
Size	Test for too few or too many characters (e.g., is social security number exactly nine digits?)
Values	Test to make sure values come from set of standard values (e.g., two-letter state codes)

Providing Feedback

- Three types of system feedback:
  - ***Status information***: keep user informed of what's going on, helpful when user has to wait for response
  - ***Prompting cues***: tell user when input is needed, and how to provide the input
  - ***Error or warning messages***: inform user that something is wrong, either with data entry or system operation

Providing Help

- Place yourself in user's place when designing help.
- Guidelines for designing usable help:
  - **Simplicity** — Help messages should be short and to the point.
  - **Organize** — Information in help messages should be easily absorbed by users.
  - **Show** — It is useful to explicitly show users how to perform an operation.

Types of Help

TABLE 11-12 Types of Help

Type of Help	Example of Question
Help on Help	How do I get help?
Help on Concepts	What is a customer record?
Help on Procedures	How do I update a record?
Help on Messages	What does "Invalid File Name" mean?
Help on Menus	What does "Graphics" mean?
Help on Function Keys	What does each Function key do?
Help on Commands	How do I use the "Cut" and "Paste" commands?
Help on Words	What do "Merge" and "Sort" mean?

Designing Dialogues and Sequence

- **Dialogue**: the sequence of interaction between a user and a system
- Dialogue design involves:
  - Designing a dialogue sequence.
  - Building a prototype.
  - Assessing usability.
- Typical dialogue between user and Customer Information System:
  - Request to view individual customer information.
  - Specify the customer of interest.
  - Select the year-to-date transaction summary display.
  - Review the customer information.
  - Leave system.

Guidelines for Designing Human-Computer Dialogues

- Consistency
- Shortcuts and Sequence
- Feedback
- Closure
- Error Handling
- Reversal
- Control
- Ease

Designing the Dialogue Sequence (Cont.)

- **Dialogue diagramming**: a formal method for designing and representing human-computer dialogues using box and line diagrams
- Three sections of the box:
  - *Top*—contains a unique display reference number used by other displays for referencing it
  - *Middle*—contains the name or description of the display
  - *Bottom*—contains display reference numbers that can be accessed from the current display

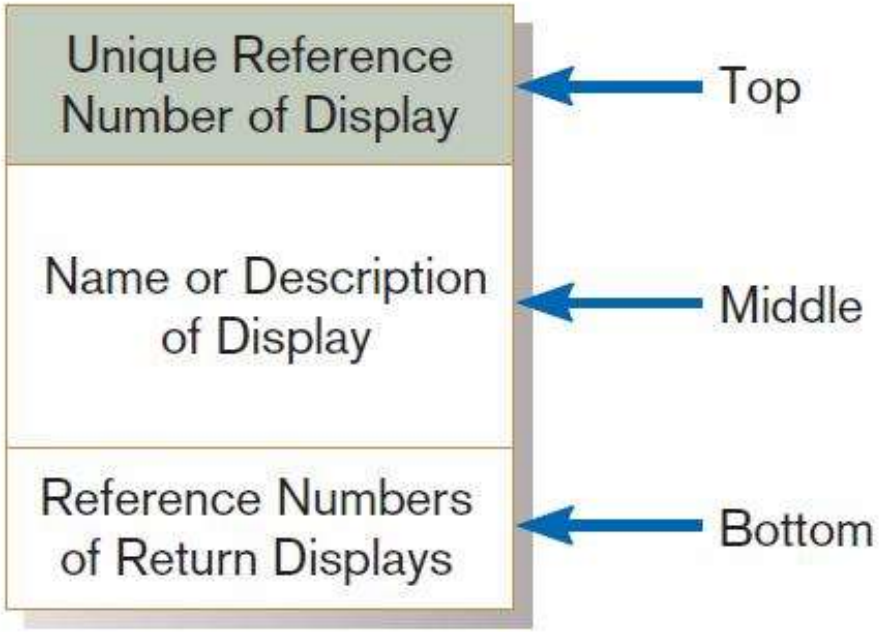


FIGURE 11-17
Sections of a dialogue diagramming box

Designing the Dialogue Sequence (Cont.)

- Dialogue diagrams depict the sequence, conditional branching, and repetition of dialogues.

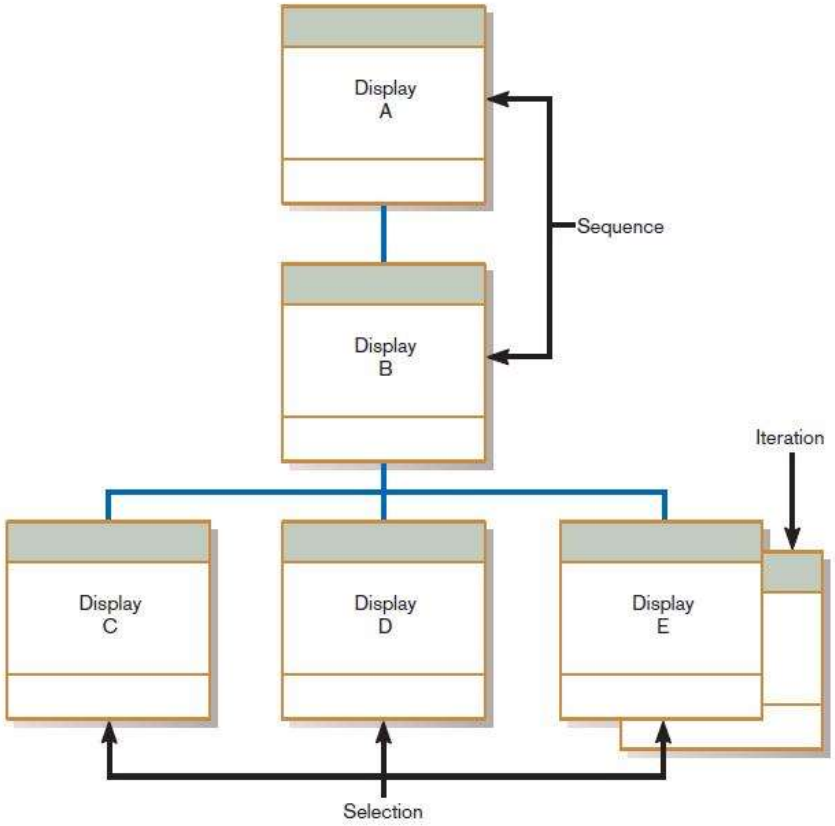


FIGURE 11-18
Dialogue diagram illustrating sequence, selection, and iteration

Building Prototypes and Assessing Usability

- Optional activities
- Building prototype displays using a graphical development environment
  - Example: Microsoft's Visual Studio .NET
  - Easy-to-use input and output (form, report, or window) design utilities

Graphical Interface Design Issues

- Become an expert user of the GUI environment.
  - Understand how other applications have been designed.
  - Understand standards.
- Understand the available resources and how they can be used.
  - Become familiar with standards for menus and forms.
- Some more graphical interface design issues:
  - #1. Prioritizing library organization over UI design. ...
  - #2. Not testing the website design enough. ...
  - #3. Inconsistent design. ...
  - #4. Focusing too strongly on standing out rather than on usability. ...
  - #5. Confusing navigation. ...
  - #6 .Too Many Words. ...
  - #7. Putting style over substance…etc.

Graphical Interface Design Issues (Cont.)

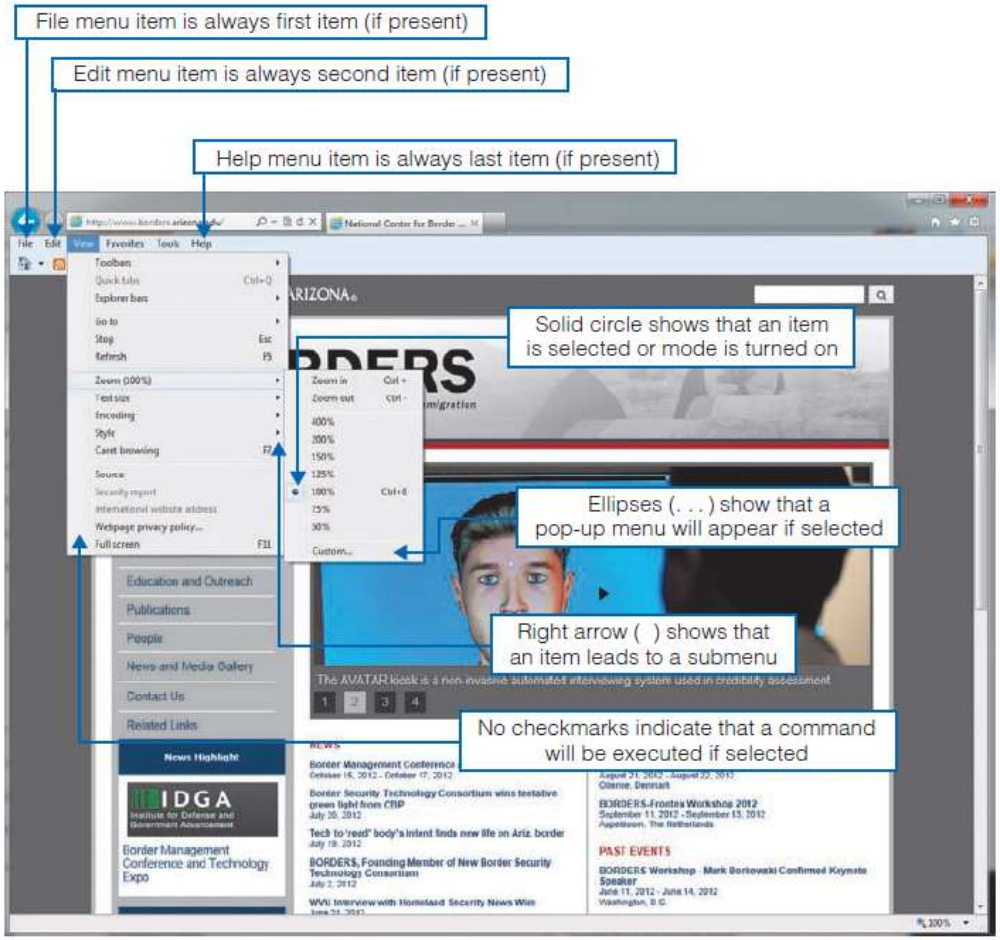


Figure 11-20
Highlighting GUI design standards (*Source:* University of Arizona.)

Electronic Commerce Application: Designing Interfaces and Dialogues for Pine Valley Furniture WebStore

- Central and critical design activity
- Where customer interacts with the company
  - Care must be put in design!
- Prototyping design process is most appropriate to design the human interface.
- Several general design guidelines have emerged.

General Guidelines

- Web's single "click-to-act" method of loading static hypertext documents (i.e. most buttons on the Web do not provide click feedback)
- Limited capabilities of most Web browsers to support finely grained user interactivity

General Guidelines

- Limited agreed-upon standards for encoding Web content and control mechanisms
- Lack of maturity of Web scripting and programming languages as well as limitations in commonly used Web GUI component libraries

Designing Interfaces and Dialogues for Pine Valley Furniture

- Key feature PVF wants for their WebStore:
  - Incorporate “menu-driven navigation with cookie crumbs” into design of interface

Menu-Driven Navigation with Cookie Crumbs

- **Cookie crumbs**: the technique of placing “tabs” on a Web page that show a user where he or she is on a site and where he or she has been
  - Allow users to navigate to a point previously visited and will assure they are not lost
  - Clearly show users where they have been and how far they have gone from home

Common Errors in Web site Design

- Opening new browser window
- Breaking or slowing down the Back button
- Complex URLs
- Orphan Pages
- Scrolling navigation pages
- Lack of navigation support
- Hidden links
- Links that don’t provide enough information
- Buttons that provide no click feedback